

claude-windows / c47f0851-d75f-45b6-9126-e48544381426

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c47f0851-d75f-45b6-9126-e48544381426\subagents\agent-a7d8c81eedaf400ca.jsonl`
- SHA-256: `f78c718dc9cf20768c0fbffd5b8829156c1be8a9869af6e5426fb3faf6e442a5`
- Source modified: `2026-06-20T08:22:45+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `subagents`
- session_id: `c47f0851-d75f-45b6-9126-e48544381426`
```

Transcript

1. user (2026-06-20T08:20:48.100Z)

Read these docs in the badminton-highlight-indexer repo and report a tight, faithful synthesis (quote exact commands/paths/risk-IDs; do not invent). I'm building the M3.5 Linux-native WASB DetectorRunner and pivoting GPU+storage from DigitalOcean to GCP (asia-southeast1 Singapore or asia-south1 Mumbai). Files:

1. docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md ? what M3.5 (NativeWasbRunner) is specified to be, its acceptance criteria, and any milestone numbering.
2. docs/DECOUPLED_COMPUTE/06-RISKS-AND-OPEN-QUESTIONS.md ? the E1?E6 risks for the native-runner build, and WASB-C3 (weight provenance).
3. docs/DECOUPLED_COMPUTE/07-COMPUTE-ABSTRACTION.md ? the compute backend abstraction (is there a ComputeBackend registry I must conform to?).
4. docs/DECOUPLED_COMPUTE/README.md and 02-COMPUTE-MAP-AND-CONTRACT.md ? the detector/runner contract.
5. docs/SETUP_NEW_MACHINE.md ? especially \$2 (GPU provisioning, doctl command), \$7b (GCS storage backend setup, [storage-gcs], GOOGLE_APPLICATION_CREDENTIALS, storage.gcs.project config), \$9 (port the box: git-archive ? scp ? mount_scratch.sh ? bootstrap.sh --gpu ? gpu_setup.sh). Quote the exact gcloud/gsutil/doctl/conda commands and any GCS-specific config keys.
6. docs/TRACKNET_WSL_SETUP.md ? the WASB conda env setup, the wasb_badminton_best.pth.tar weights, how wasb_infer.py is run, the WASB-SBDT repo layout (where configs/, src/, pretrained_weights/ live), and the device assumptions.
7. deploy/digitalocean/gpu_setup.sh, deploy/digitalocean/bootstrap.sh, deploy/digitalocean/mount_scratch.sh ? what they do, what env vars / paths they set up (WASB repo dir, conda env name, weights path, scratch mount), and whether they fetch the WASB weights.

Report as sections per topic. Be precise about: the exact conda env name, the WASB-SBDT repo path on a Linux box, where weights land, how device is selected, and any env vars the native runner should read (e.g. weights path, repo dir, python bin, device). Also flag any place the docs say "DigitalOcean GPU droplet in sgp1" that is now contradicted by the GCP pivot (DO has no Singapore GPU).

2. assistant (2026-06-20T08:20:50.600Z)

I'll read these docs systematically to give you exact commands, paths, and risk flags for your M3.5 Linux-native WASB DetectorRunner pivot to GCP.

3. user (2026-06-20T08:20:51.090Z)

```
1      # 04 . Build & test plan
2
3      **Status:** PROPOSAL . **Date:** 2026-06-20
4
5      > A staged plan a solo dev can stop at any milestone with a working, tested artifact. It ships the
6      > **container-clean path first** (Gemini + torchvision + motion + Gen-0) and defers the WSL-bound
```

```

7      > WASB/TrackNet detectors until a Linux-native runner exists ? those two are the only
HIGH-risk components
8      > in the stack (see [02](02-COMPUTE-MAP-AND-CONTRACT.md)).
9
10     ## 1. Target architecture (diagram-in-words)
11
12     ```
13     CONTROL PLANE (thin, cheap; already shipped as the async job model)
14     - validates API key, accepts {job_type, uri}; returns 202 + job_id; GET /jobs/{id}
15     - holds NO video bytes; enqueues, polls
16         ? job descriptor (bucket keys)           ? status / cost
17         ?                                           ?
18     COMPUTE WORKER (rented GPU; container OR venv; pay GPU-seconds)
19     - ENTRYPOINT dispatcher: mode ? {train, infer}
20     - sync-first: rclone cp bucket ? /scratch (local NVMe); decode/detect on GPU
21     - write artifacts ? /scratch ? upload to bucket; checkpoint train ?30 min
22     - record usage (gpu_seconds, egress, gemini_calls) ? cost ledger (05)
23         ? pull inputs                             ? push outputs
24         ?                                           ?
25     OBJECT-STORE BUCKET (data plane ? bytes never via the control plane)
26     videos/  labels/  manifest.json  weights/  outputs/
27     ```
28
29     The control plane coordinates metadata + jobs; the heavy bytes + GPU work live in
the worker; the
30     bucket is the only shared state. This is the control/data-plane split
[PLATFORM_ARCHITECTURE.md](../PLATFORM_ARCHITECTURE.md)
31     already names ? what's new is the container/VM image, the real bucket, the rented GPU,
and the cost ledger.
32
33     ## 2. The two entrypoints (one dispatcher; container or `run.sh` on a VM)
34
35     train ? *bucket of videos + labels ? versioned weights back to the bucket:*
36     1. Look for the latest checkpoint under `weights/checkpoints/`; resume if present
(spot-safety +
37     the firm idempotency/resumability requirement ? a 40-min job must not restart on a
blip).
38     2. `rclone copy` the corpus to `/scratch` (sync-first).
39     3. Run the Gen-0 harness as-is ? already a single self-contained command producing a
versioned
40     artifact: `python -m training.gen0.harness --manifest <?> --floor <?> --version
<semver>`. Gen-0 needs
41     only numpy/scipy (~4.5 GB VRAM, minutes). Gen-2 VLM is explicitly out of scope**
([06](06-RISKS-AND-OPEN-QUESTIONS.md)).
42     4. Run the product-side ship-gate as pre-flight
([`backend/eval/regression.py`](../backend/eval/regression.py)
43     + `metrics.match_intervals`): no model admits itself to serving.
44     5. Upload weights + manifest to `weights/<semver>/` (ADR-008 artifact bundle,
content/semver-keyed).
45     Checkpoint every ?30 min.
46
47     infer ? *new upload ? segments/highlights ? outputs back to the bucket:*
48     1. `rclone copyto $VIDEO_URI /scratch/in.mp4` ? sync-first, not a FUSE random-access
decode.**
49     2. Compute whole-file MD5 ? `video_id` (existing `compute_video_id`). Store the byte-
identical file the
50     labels were made against ? any re-encode changes the MD5 and breaks the join
([02](02-COMPUTE-MAP-AND-CONTRACT.md)).
51     3. Run the segmenter (`gemini` default; `yolo_hybrid`/`motion` offline). Gemini path
uploads ?500 MB chunks.
52     4. Write `outputs/<video_id>/{highlights.mp4, intervals.csv (decimal-second
[start,end]), report.json}`.
53     5. Inference is checkpoint-free ? on spot preemption just retry; make the front-end
idempotent.
54

```

```

55     ## 3. Bucket layout
56
57     ```
58     rally-bucket/
59     ??? videos/<md5>.mp4          # immutable, byte-identical to what labels were made
on
60     ??? labels/<sport>/<video_id>.csv # collector-exported start,end CSVs
61     ??? manifest.json             # the join table (video_id, csv, local_path, md5,
license, fps?)
62     ??? weights/
63     ?   ??? <semver>/model + manifest # train output (artifact bundle)
64     ?   ??? checkpoints/             # spot-resume state (model + optimizer + step)
65     ??? outputs/<video_id>/{highlights.mp4, intervals.csv, report.json}
66     ```
67     Auth via env/secret (`AWS_ACCESS_KEY_ID/SECRET` for S3/R2/Spaces, or a mounted GCS
service-account JSON) ?
68     never baked into the image. Keep `is_commercial`/CURATED training rows physically
separated; only those
69     export by default.
70
71     ## 4. Milestone ladder (each with a concrete acceptance test)
72
73     **M0 ? Local CPU smoke test (`$0`, no GPU, no provider).**
74     Build the torch image *or* a venv; run the **committed synthetic, video-free regression
fixtures**
75     ([`golden_fixtures.py`](../../backend/eval/golden_fixtures.py)) + motion/yolo_hybrid on
a tiny sample,
76     CPU-only (torchvision auto-falls-back).
77     *Accept:* `infer --video samples/clip.mp4` produces an `intervals.csv`, and the in-repo
regression subset
78     passes ? no GPU, no WSL. (Runnable in GitHub Actions.)
79
80     **M1 ? Single GPU, manual run.**
81     Launch one GPU (DO Paperspace/Gradient A4000 or GCE Spot L4); run the worker.
82     *Accept:* `nvidia-smi` + `torch.cuda.is_available()==True` inside the worker, and one
`infer` on a real
83     clip completes on GPU, faster than the M0 CPU run.
84
85     **M2 ? Train-from-bucket.**
86     Create the bucket; populate `videos/`, `labels/`, `manifest.json` (**incl. the `md5`
field ? prerequisite,
87     [02](02-COMPUTE-MAP-AND-CONTRACT.md) $4**). Run `train`, pulling via `rclone`, running
Gen-0, pushing a
88     versioned bundle back.
89     *Accept:* `weights/<semver>/` appears in the bucket, the artifact passes the ship-gate
pre-flight, and a
90     re-run **resumes from checkpoint** rather than restarting.
91
92     **M3 ? Inference endpoint on new upload (+ per-video billing).**
93     Deploy as a serverless endpoint (Cloud Run L4 / RunPod Serverless). Control plane
returns `202 + job_id`;
94     worker pulls the upload, writes `outputs/<video_id>/`, and records the cost row
([05](05-COST-ATTRIBUTION.md)).
95     *Accept:* `POST {job_type:"infer", video_uri:"?"}` with a valid API key returns a
`job_id`; polling yields
96     `highlights.mp4` + `intervals.csv` in the bucket **and a per-video cost**; a bad/absent
key is rejected.
97
98     **M3.5 ? WSL?native detector parity, tested on DigitalOcean (the "WSL runs seamlessly"
gate).**
99     Stand up a **`LinuxNativeRunner`** behind the existing `DetectorRunner` ABC
([`detectors/base.py`](../../backend/pipeline/detectors/base.py))
100     ? **"the main de-risk goal"**: it **skips all `wsl`/`conda activate`/staging**, loads the
WASB/TrackNet weights
101     natively (torch/TF), and writes the **identical `Frame,Visibility,X,Y` CSV** so the

```

```

model-agnostic
102     `parse_trajectory_csv` + `trajectory_to_action_windows` (and everything downstream ?
windowing, candidates,
103     AI handoff, eval) run verbatim, byte-for-byte. On a DO GPU Droplet / Paperspace
GPU (Ubuntu +
104     NVIDIA + CUDA) you replicate the `TRACKNET_WSL_SETUP.md` conda env natively, minus
WSL ? and since WASB
105     ships pretrained badminton weights (`wasb_badminton_best.pth.tar`), you get a real
trajectory on DO
106     today, no TrackNet training needed. Local Windows dev is untouched ? the WSL
adapters stay as one
107     backend of the ABC (the base guarantees "existing behavior is 100% preserved"); env/OS
selects the runner.
108     *This milestone doubles as the [ `DESKTOP_APP_PLAN.md` ](../DESKTOP_APP_PLAN.md) P0
compute-feasibility spike.*
109     *Accept (the seamlessness guarantee):* a parity test ? the same clip through the
WSL backend (Windows)
110     and the LinuxNativeRunner (DO) yields `Frame,Visibility,X,Y` CSVs that match
within tolerance and
111     identical candidate windows; the WASB/TrackNet hybrid segmenter produces the same
rallies on DO as on the
112     WSL box. (WASB and TrackNetV4 are torch vs TF ? keep them as separate runners/images
sharing `/scratch`;
113     never co-install.)
114
115     M4 ? Scale-to-zero / cost-guard.
116     Confirm the endpoint scales workers to zero when idle; add a per-key rate limit + a
provider hard spend cap.
117     *Accept:* after a burst, the dashboard shows $0 idle GPU billing between jobs; a
synthetic flood on one
118     leaked key is throttled; the spend alert fires below $200 in a forced-overflow test.
119
120     ## 5. Smallest first PR
121
122     PR: "Add bucket-aware I/O shim + `train`/`infer` dispatcher + (optional) `rally-torch`
Dockerfile ?
123     CPU-clean path only." In the indexer repo:
124     1. A thin storage shim: `fetch(uri)?local_path` + `put(local_path, uri)` (over
`rclone`/`aws cp`),
125     wired behind the existing `resolve_video_path`/`validate_input_path` so local paths
still work unchanged.
126     2. A `run.sh` (or `python -m`) dispatcher (`mode ? {train, infer}`) calling the
existing CLI entry
127     points ? no pipeline-logic changes.
128     3. De-harden the "output" literals in
[ `trajectory_hybrid.py:196,272` ](../../backend/pipeline/segmenters/trajectory_hybrid.py)
129     to honor an output root (small, isolated).
130     4. (Optional) a `Dockerfile` for the torch image (pytorch CUDA 12.4 base, `apt
ffmpeg`, torch from the
131     explicit `--index-url`, `supervision<0.30`, pre-baked COCO weights, `GEMINI_API_KEY`
via env).
132     5. CI: run the video-free regression subset (on CPU, in a container or venv) ?
this is the M0 gate.
133
134     This PR is intentionally WSL-free, GPU-optional, and touches no detector internals ? it
lands the
135     container/bucket seam (the two things missing today) while deferring every HIGH-risk
component to a later,
136     isolated PR.
137
138     ## 6. Container image strategy (when you do containerize)
139
140     - Image A ? `rally-torch` (ship first; M0?M3): base
`pytorch/pytorch:2.5.x-cuda12.4-cudnn9-runtime`;
141     `apt ffmpeg`; torch/torchvision from `--index-url ?/cu124`; `supervision<0.30`; pre-

```

```

baked COCO weights;
142     `GEMINI_API_KEY` as runtime secret. Covers gemini, motion, yolo_hybrid.
143     - Image B ? `rally-tracknet-tf` (M4+): base `tensorflow/tensorflow:2.16-gpu`. This
image is what
144     kills the WSL bridge ? inside Linux, TF-GPU runs natively. Lift the TrackNet/WASB
inference out of
145     WSL into an in-container module reading frames from `/scratch`; vendor the
research repos + weights;
146     replace hardcoded `device='cuda'` with the Linux-native `DetectorRunner` the ABC
already anticipates;
147     keep the WSL bridge as one backend of a two-backend interface so Windows dev +
existing tests stay green.
148     Torch and TF talk over a shared `/scratch` volume ? never co-install TF + torch in
one image (segfault).
149

```

4. user (2026-06-20T08:20:51.875Z)

```

1     # 06 · Risks & open questions
2
3     Status: PROPOSAL · Date: 2026-06-20
4
5     > The technical plan is cheap and de-risked. The gates are mostly scope, legal, and
budget-framing ?
6     > plus a few staleness checks. This doc is the adversarial review (a completeness critic
ran over the build
7     > plan) distilled into risks + the decisions only the owner can make.
8
9     ## 1. Risks (ranked)
10
11     1. WSL2 bridge is the load-bearing technical risk ? but the codebase is already built
to drop it. Both
12         high-accuracy detectors (WASB, TrackNet) run only via `wsl ? conda activate` with
hardcoded `~/models`
13         paths, hardcoded `device='cuda'`, un-vendored repos, and conflicting TF-vs-torch
runtimes. The
14         Linux-native path is plausible but unproven ? it has never been run, and shipping
a model commercially
15         is gated on weight-licensing (WASB-C3). What's already in our favor: the
`DetectorRunner` ABC
16         ([`detectors/base.py`](../../backend/pipeline/detectors/base.py)) was built
expressly for this swap ?
17         WSL plumbing lives in a separate `WslCommandMixin` ("a future Linux-native runner
must not inherit WSL
18         plumbing"), the base preserves "existing behavior 100%" for Windows dev, and any
runner emitting the
19         same `Frame,Visibility,X,Y` CSV slots in verbatim. Mitigation: ship the
torchvision/Gemini/Gen-0 path
20         first (M0?M3); prove the native runner via the M3.5 WSL?native parity test on
DigitalOcean (DO GPU =
21         Ubuntu+CUDA = the native conda env, minus WSL; WASB's shipped badminton weights make
a real trajectory
22         testable today). That parity test IS the "WSL runs seamlessly" guarantee, and
doubles as the
23         DESKTOP_APP_PLAN P0 feasibility spike. Do not let the prose oversell this as done
until parity is green.
24     2. TF + torch in one image segfaults. Mitigation: two single-framework images
sharing `/scratch`.
25     3. The $200 envelope only holds because the Gen-2 VLM is excluded. What fits in $200
is the
26         container/bucket/Gen-0 plumbing + inference, not the owned-VLM retrain the
North Star ultimately
27         wants (6?8-GPU GRPO reference, "largest open risk, currently unquantified"). State
this plainly to
28         anyone reading the burn-down; don't conflate "decoupled train+infer on $200" (true

```

for the cheap

29 classifier) with "train the VLM on \$200" (likely false).

30 4. ****Free-credit reality.**** Cost figures are ****approximate ? verify before committing.****

RunPod's "\$200

31 free" is a myth (~\$5). Every "\$200 tier" has a catch: ****GCP \$300/90-day is the only**

clean win**

32 is split/earned; Azure \$200 is 30-day. ****You have real DO \$200**** ? but DO lacks

serverless-GPU metering

33 (see the budget-vs-billing tension, [03](03-PLATFORM-OPTIONS.md) §4).

34 5. ****Data-contract break via re-encode.**** MD5 keys the whole 3-repo join; the

collector's `nvenc`/`libx264`

35 proxy transcode changes it, and the export `manifest.json` ****omits `md5`.****

Mitigation:** *add `md5` to**

36 the collector export + decide original-vs-proxy keying ? an M2 prerequisite**, not a

footnote.

37 6. ****Spot preemption loses local disk.**** ***Mitigation:** checkpoint train state to the

bucket every ?30 min;

38 inference jobs are retry-safe.

39 7. ****Gemini key custody / cost.**** Default segmenter silent-fails without the key

(~\$0.05/run). ***Mitigation:**

40 inject as a secret; rate-limit per API key so a leak can't drain the budget or

someone else's bill.

41

42 **## 2. Legal / scope gates (these can *block*, not just delay)**

43

44 - ****Scope re-ratification.**** The ratified posture is ****rent nothing until the owned**

model has a

45 ship-gate-passing reason" (`VIDEO\_LOCALITY\_MODEL.md`), and the harness is ****single-**

tenant / local-first

46 ? explicitly NOT a service" (`EXPERIMENT\_HARNESS.md`). ****The North Star requires**

explicitly overriding

47 both** ? an owner decision, ideally recorded in

`docs/archives/decisions/DECISIONS.md`.

48 - ****DPA / no-train gate on the *inference-on-uploads* half specifically.**** Plans forbid

sending non-consented

49 video to cloud for training (****only owned/consented footage, never user uploads****).

Inference on a

50 user's upload over a no-train DPA (Gemini via Vertex, zero-retention) is generally

fine, ****but a**

51 non-owner user uploading footage to your cloud raises consent/processing-DPA questions

that must be

52 answered before M3.** Treat this as a ****gating precondition on the upload-inference**

half**, not a footnote.

53 - ****Weight/dataset licensing.**** WASB-C3 checkpoint provenance + per-checkpoint weight

terms gate shipping any

54 cloud-trained model commercially (orthogonal to compute portability, but real for the

TF-image/M4 epic).

55

56 **## 3. Staleness checks before building (verified where noted)**

57

58 - ? ****Async control plane exists**** (`jobs` table, `job\_worker.py`) ? confirmed

2026-06-20.

59 - ? ****Committed synthetic video-free regression fixtures exist**** (`golden\_fixtures.py`)

? M0 is a real \$0 gate.

60 - ? ****"output" still hardcoded**** (`trajectory\_hybrid.py:196,272`) ? still a live

blocker.

61 - ? ****Gen-0 harness exists**** (`training/gen0/harness.py`).

62 - ? ****Confirm `DEFERRED\_HARDENING\_PLAN.md` #16**** specifies exactly what the control

plane shipped vs deferred

63 (queue tech is still undecided: Arq vs Postgres-queue vs Hatchet vs Temporal) ? not

harvested in this pass.

64 - ? ****Confirm the collector export manifest `md5` gap**** at build time and pick the

keying rule (\$5 risk).

65

66 **## 4. Decisions for the owner**

```

67
68     | # | Decision | Recommendation |
69     |---|---|---|
70     | 1 | **Greenlight this as an active project** (override the "rent nothing / not a
service" posture for a scoped build)? | Yes, scoped to **inference + Gen-0 train only**; record
in DECISIONS.md. |
71     | 2 | **Where does the inference endpoint run** (per-video billing vs D0 credits)? | D0
for M1?M2 dev/train; **GCP Cloud Run GPU or RunPod Serverless for M3** (native per-invocation
metering = the per-video bill). Or single-vendor GCP ($300 trial) for all. |
72     | 3 | **Bucket:** D0 Spaces (1 TiB egress incl., uses credits) vs R2 (zero egress) vs
GCS (native if GCP)? | D0 Spaces for the dev loop; revisit R2 if reel egress grows. |
73     | 4 | **Docker now or VM/venv first?** | venv/SkyPilot for M0?M2; container at M3
(serverless needs it). |
74     | 5 | **DPA/consent posture for non-owner uploads** (gates M3)? | Resolve before M3;
inference-only no-train DPA + consent capture. |
75     | 6 | **`md5` in collector export + original-vs-proxy keying** (gates M2)? | Add `md5`;
key on the byte-identical file labels were made on. |
76     | 7 | **Confirm the VLM is out of scope for this $200 build?** | Yes ? Gen-2 is a
separate, separately-budgeted effort. |
77     | 8 | **Ratify the M3.5 WSL?native parity-on-DigitalOcean approach** (drop WSL on Linux,
keep it for Windows dev, gate on a CSV parity test)? | Yes ? it's the "WSL runs seamlessly"
guarantee and doubles as the DESKTOP_APP_PLAN P0 spike. ? **Resolved 2026-06-20: $200 credit
covers Paperspace GPUs + a free unbilled M4000** (so the parity run is $0; verify the M4000's
`sm_52`/Maxwell arch is in your pinned torch build). |
78
79     ## 5. What success looks like
80
81     A non-owner user `POST`s a video (with an API key), gets back a highlight reel + rally
CSV **and a
82     line-item cost for that video**; the owner runs `train` over a bucket and gets a
versioned, ship-gated model
83     plus the run's cost segmented per contributing video ? **all on rented, scale-to-zero
GPU, for well under
84     $200, with zero idle-GPU billing**, and with the WSL bridge either gone (M4) or cleanly
deferred behind the
85     container-clean path.
86

```

5. user (2026-06-20T08:20:52.573Z)

```

1     # 07 · Compute abstraction ? segmenting GPU vs CPU + a spec-aware device layer
2
3     **Status:** PROPOSAL (design) · **Date:** 2026-06-20 · code-verified 2026-06-20
4
5     > **"Do we have a GPU Runner interface? How do we segment GPU vs CPU work ? cleanly
abstracting the GPU while
6     > leaving room for customizations based on the GPU spec?" Answer: **not yet as a single
interface**, but the
7     > seam and a spec-probe already exist. This doc defines the missing **device layer** and
the **GPU/CPU
8     > segmentation rule** ? small, and consistent with the codebase's ABC+Registry idiom.
9
10    ## 1. What exists today (honest inventory)
11
12    | Concern | Today | Verdict |
13    |---|---|---|
14    | Global GPU switch | `use_gpu: bool = True`
([`config/models.py:119`](../../backend/config/models.py)) | Coarse ? one flag for the whole
run. |
15    | torchvision person detector | `self.device = "cuda" if use_gpu and
torch.cuda.is_available() else "cpu"` + warn/fallback
([`person_detector.py:74-76`](../../backend/pipeline/detectors/person_detector.py)) | **Clean
fallback, but no spec-awareness** (no VRAM/batch/precision logic; per-frame `to(device)`). |
16    | WASB detector | hardcoded `device='cuda'`, **no CPU fallback** | Inconsistent ? a
portability/robustness gap. |

```

```

17 | TrackNet (TF) | GPU only via WSL; healthcheck requires a visible GPU | WSL-bound (see
[02](02-COMPUTE-MAP-AND-CONTRACT.md), [04 M3.5](04-BUILD-AND-TEST-PLAN.md)). |
18 | `DetectorRunner` ABC | abstracts the task (run_predict/healthcheck)
([`detectors/base.py`](../backend/pipeline/detectors/base.py)) | Device handling lives
inside each concrete runner, not shared. |
19 | `ComputeBackend` (planned) | abstracts where a job runs (local/hosted/byo_worker)
| Orthogonal to how GPU work executes ? don't conflate. |
20 | GPU spec probe | `gpu_name()` + `gpu_stats()` (name, VRAM used/free via `nvidia-smi`)
([`run_telemetry.py`](../backend/utlis/run_telemetry.py)) | The seed of a `DeviceSpec` is
already here. |
21 | Macro GPU/CPU split | detect-once-on-GPU ? persist CSV/features ? re-score on CPU
(EXPERIMENT_HARNESS, GOLDEN_DATA) | Already the right segmentation ? at the artifact
boundary. |
22
23 Conclusion: there is no unified "GPU Runner." Device selection is scattered across
components with three
24 different styles (global flag, clean-fallback, hardcoded-cuda). The fix is one small
shared layer.
25
26 ## 2. Two orthogonal axes ? keep them separate
27
28 ```
29 WHERE the job runs          ?   HOW a GPU stage executes on this box
30 (ComputeBackend ? coarse,   ?   (DeviceContext ? fine, spec-aware,
31 per-job: local | DO droplet ?   per-stage: probe the actual GPU ?
32 | serverless | byo_worker) ?   resolve batch/precision/chunk knobs)
33 picks the machine ???????? on that machine, DeviceContext.detect()
34                               ?   probes the real GPU and tunes the work
35 ```
36
37 `ComputeBackend` answers which machine; DeviceContext answers given this
machine's GPU, how do I run
38 this stage. A serverless L4, a DO RTX-4090 Droplet, and CPU CI each get correct knobs
automatically,
39 with per-spec overrides available. Conflating the two is the main design trap.
40
41 ## 3. The GPU/CPU segmentation rule
42
43 Classify every pipeline stage; GPU work is concentrated in the detection substrate.
44
45 | Stage | Class | Notes |
46 |---|---|---|
47 | Validators (format/res/pts/scene/blur/relevance) | CPU | ffmpeg + OpenCV/numpy |
48 | ffmpeg chunk extraction / stitching | CPU (`libx264`); GPU optional (`nvenc`) | no
driver-codec dependency by default |
49 | Person detection (torchvision Faster R-CNN) | GPU (auto CPU-fallback) |
`DeviceContext` consumer |
50 | ByteTrack tracking | CPU | runs on the detections |
51 | Shuttle trajectory (WASB torch / TrackNet TF) | GPU | `DeviceContext`
consumer; the WSL?native target |
52 | Windowing / candidate merge | CPU (numpy) | model-agnostic |
53 | AI handoff (Gemini) | network I/O | no local GPU; `ai_max_workers` parallelism |
54 | DB writes, eval/scoring, calibration | CPU (numpy) | device-agnostic |
55 | Gen-0 head training | tiny GPU/CPU | ~4.5 GB, minutes |
56
57 The rule: a GPU-bound stage is a pure, restartable unit whose only job is `frames ?
artifact` (the
58 `Frame,Visibility,X,Y` CSV or a feature JSON). Everything downstream is device-agnostic
CPU that reads that
59 artifact. This single principle delivers three things at once:
60 1. Clean GPU/CPU segmentation ? only the detection substrate touches a
`DeviceContext`.
61 2. Cross-machine decoupling (the North Star) ? the artifact boundary is exactly
where GPU and CPU work
62 can live on different machines (it's already how EXPERIMENT_HARNESS / GOLDEN_DATA

```



```

split today).
63     3. **Per-stage billing** ([05](05-COST-ATTRIBUTION.md)) ? GPU-seconds are attributable
to the GPU stage.
64
65     ## 4. The proposed device layer (the "GPU Runner")
66
67     Two dataclasses + one context object ? injected into GPU stages, **replacing** scattered
`device=` logic.
68
69     ```python
70     @dataclass(frozen=True)
71     class DeviceSpec:                                # WHAT hardware we got (probe once, cache)
72         kind: str                                    # "cuda" | "mps" | "cpu"
73         name: str | None                             # "NVIDIA L4" ? from run_telemetry.gpu_name()
74         vram_gb: float | None                         # from gpu_stats() memory.free/used
75         compute_capability: tuple[int,int] | None
76         supports_fp16: bool
77         supports_bf16: bool
78         num_devices: int
79
80     @dataclass
81     class DeviceKnobs:                                # HOW to run, resolved from spec (§5)
82         batch_size: int
83         precision: str                                # "fp32" | "fp16" | "bf16"
84         chunk_duration_s: float
85         frame_skip: int
86         max_inflight_chunks: int
87         ai_max_workers: int
88
89     class DeviceContext:                              # the GPU Runner ? injected into GPU stages
90         spec: DeviceSpec
91         knobs: DeviceKnobs
92         def to(self, model_or_tensor): ...            # placement (cuda/mps/cpu)
93         def autocast(self): ...                      # precision ctx (no-op on cpu)
94         def run_with_oom_retry(self, fn, halve="batch_size"): ... # CUDA-OOM ? smaller
batch
95         @classmethod
96         def detect(cls, config) -> "DeviceContext":  # probe spec ? resolve profile (§5)
97             ...
98     ```
99
100     - **One probe, reused:** `DeviceSpec` is built from the `nvidia-smi` data
`run_telemetry` already collects ?
101     so the spec also feeds the **cost ledger's `gpu_type`** ([05](05-COST-ATTRIBUTION.md))
and the run-log
102     header. One source of truth for "what GPU am I on."
103     - **Graceful by construction:** no CUDA ? a `cpu` spec + conservative knobs. This is
where **WASB's
104     hardcoded `device='cuda'` gets fixed ? it becomes `ctx.to(model)`.
105     - **`DetectorRunner` keeps its task contract;** the device is injected (constructor or
call), so the
106     `LinuxNativeRunner` ([M3.5](04-BUILD-AND-TEST-PLAN.md)) is just **a `DetectorRunner`
whose `DeviceContext`
107     has a native CUDA spec** ? the same abstraction serves WSL, native-Linux, and CPU.
108
109     ## 5. Spec-aware customization ? a `DeviceProfile` registry
110
111     The "leave scope for customizations based on the GPU spec" requirement = a registry
(same ABC+Registry idiom
112     as segmenters/providers/validators) that maps a `DeviceSpec` ? tuned `DeviceKnobs`.
113
114     ```python
115     class DeviceProfile(ABC):
116         name: str
117         def matches(self, spec: DeviceSpec) -> bool: ...

```

```

118         def knobs(self, spec: DeviceSpec, overrides: dict) -> DeviceKnobs: ...
119
120     # Built-ins (illustrative): CpuProfile, T4_16GB, L4_24GB, A4000_16GB, RTX4090_24GB,
A100_40GB ?
121     # + HeuristicProfile: derive batch_size/precision from vram_gb + compute_capability when
the
122     #   exact GPU name isn't in the table (so an unknown GPU still runs sensibly).
123     ...
124
125     **Resolution order (most specific wins):**
126     1. **Explicit override** ? `config.indexing.device.{batch_size,precision,?}` or a CLI
flag (`--batch`, `--precision`). *Full manual control.*
127     2. **Matched named profile** ? e.g. an `L4_24GB` profile (bf16, batch N, chunk T).
128     3. **Heuristic from `DeviceSpec`** ? derive from VRAM + compute capability when the name
is unknown.
129     4. **Conservative CPU defaults** ? when there's no GPU.
130
131     So the GPU is **cleanly abstracted** (stages call `ctx.to()/ctx.autocast()/ctx.knobs`,
never `torch.cuda`
132     directly) **and** customizable per spec (a config override tunes batch/precision/chunk
for a 4090 vs an L4).
133
134     > **Scoping (owner decision 2026-06-20): a small *provider-grounded* set + heuristic
fallback.** Curate
135     > profiles only for GPUs your providers actually rent, and fall back to a **VRAM
heuristic** for everything
136     > else. This flips the earlier YAGNI worry for one reason: **the cost ledger
([05](05-COST-ATTRIBUTION.md))
137     > needs the *same* `{provider, gpu_type} ? $/hr` table for billing anyway**, so the set
isn't speculative ?
138     > billing requires it and the device knobs ride along on the same rows. The provider-
grounded set:
139     >
140     > | Tier | GPUs | Notes |
141     > |---|---|---|
142     > | **D0 / Paperspace (now)** | **M4000 8GB (free tier)** . P4000/RTX4000 . A4000 16GB .
A5000/A6000 . L40S . A100 | M4000 is the unbilled Gradient-plan card (Maxwell `sm_52` ? verify
torch build); credit covers the rest |
143     > | **GCP (later)** | T4 16GB . L4 24GB (Cloud Run GPU) . A100 | T4/L4 are GCP/AWS,
**not** D0 |
144     > | **RunPod/Vast (only if used)** | RTX 4090 24GB | **4090 is not on D0 or GCP** ? add
only if serverless-on-RunPod |
145     > | **fallback** | anything else | `HeuristicProfile` derives knobs from VRAM + compute
capability |
146     >
147     > Two caveats: (1) a named entry exists for **identity + $/hr rate first**, tuning
second ? keep its perf
148     > knobs **= the heuristic until a benchmark shows a per-GPU win** (the workload is
inference-light, batch?1,
149     > so tuning may never matter). (2) `DeviceContext` itself is **not urgent**: WASB's
hardcoded
150     > `device='cuda'` only bites when the native runner lands, so **build it at
[M3.5](04-BUILD-AND-TEST-PLAN.md),
151     > not at M0.**
152
153     ## 6. Smallest first step (do this at M3.5, not M0)
154
155     1. Add `DeviceSpec` + `DeviceContext.detect()` from the existing `run_telemetry` probes;
ship a
156     `CpuProfile` + a `HeuristicProfile` (derive batch/precision from VRAM) **+ a
config/CLI override**, plus
157     the small **provider-grounded** set (D0/Paperspace now, GCP later ? $5) whose tuning
knobs **start equal
158     to the heuristic** (they exist for identity + the cost-ledger rate, not tuning, until
measured). Defaults

```

```

159     reproduce today's behavior (no-op until adopted).
160 2. Route **WASB** through it when you build the native runner ([M3.5](04-BUILD-AND-TEST-
PLAN.md)) ? this is
161     the consumer that justifies the layer (it removes the hardcoded-`cuda` footgun). The
torchvision person
162     detector can follow (it already has clean fallback, so it's optional cleanup).
163 3. Feed `spec.name`/`spec.vram_gb` into the cost ledger's `gpu_type` ([05](05-COST-
ATTRIBUTION.md)) ? one
164     probe, two uses.
165
166 This lands the unified GPU abstraction without touching the windowing/eval/CPU paths ?
they stay
167 device-agnostic by design ($3). **Add named `DeviceProfile`s only when a benchmark shows
per-GPU tuning
168 pays.**
169

```

6. user (2026-06-20T08:20:53.431Z)

```

1     # Decoupled Compute ? Cloud-Native GPU for Train + Inference
2
3     **Status:** ? PROPOSAL / research ? for owner review. **No code yet.**
4     **Date:** 2026-06-20 . **Owner:** avidullu
5     **Lifecycle:** *Critical active project.* When delivered, **move this whole directory to
`docs/archives/past_projects/`** (keeps the strategy tree clean).
6
7     > **What this directory is.** A single, unified place that answers one question: *can we
lift the
8     > rally-segmentation pipeline off the owner's local Windows/WSL box and run it on a
**rented,
9     > credit-budgeted, cloud-native GPU** ? driven by an API key ? that does **both** (a)
**retrain** the
10    > rally model from a **bucket of videos** and (b) run **inference on new uploads** ? and
**bill each
11    > video / each training run exactly**, including for a non-owner user?* Short answer:
the *design*
12    > already exists across our strategy docs; the *build* (a container/VM image, a real
bucket, a chosen
13    > provider, a cost ledger) does not. This directory inventories what's planned, maps the
real compute,
14    > compares platforms, and lays out a cheapest-first build & test plan that fits the
**$200** budget.
15
16    ---
17
18    ## North Star
19
20    A **decoupled** pipeline where the **control plane** (a thin API that accepts jobs and
reports status)
21    is separated from the **data/compute plane** (the heavy bytes + GPU work), which runs in
a **rented
22    cloud GPU** reading from and writing to an **object-store bucket**. Two verbs:
23
24    - **`train`** ? consume a bucket of videos + labels ? produce versioned model weights
back to the bucket.
25    - **`infer`** ? a new uploaded video ? rally segments + highlight reel + an eval report
back to the bucket.
26
27    ?with **exact per-video (inference) and per-run (training) cost attribution** so the
cost can be
28    charged to whoever ran the job.
29
30    ---
31
32    ## TL;DR (read this, then dive into the numbered docs)

```

```

33
34     1. **Yes ? decoupling is already designed, in depth.**
[PLATFORM_ARCHITECTURE.md](../PLATFORM_ARCHITECTURE.md)
35     is a control-plane/data-plane split with a `ComputeBackend` registry (`local | hosted
| byo_worker`)
36     and a `StorageBackend` registry (S3/GCS/Azure via `fsspec`);
[HOSTING_PLAN.md](../HOSTING_PLAN.md)
37     already names **GPU box = first `byo_worker`** with GCS buckets + signed-URL uploads;
$5A specifies
38     **training on a rented GPU** via `ms-swift`. ? **[01](01-EXISTING-PLANS-AND-
GAP.md)**.
39     2. **What's missing is everything that turns those seams into the North Star:** a
container/VM image,
40     a **chosen** provider + an **API-key/provisioning** flow, a **real bucket** (today
it's OneDrive
41     auto-sync ? a proto-bucket), a **download-to-local** shim, path de-hardcoding, a
**bucket-triggered
42     retrain**, a **remote inference endpoint**, and **cost metering**. ?
**[01](01-EXISTING-PLANS-AND-GAP.md)**, **[02](02-COMPUTE-MAP-AND-CONTRACT.md)**.
43     3. **$200 is comfortable ? for the `right` scope.** It easily covers the **inference
pipeline + Gen-0
44     head training** (numpy-class, trains in minutes on a small GPU) decoupled onto one
cheap GPU. It does
45     **NOT** cover the **Gen-2 VLM fine-tune** (ms-swift/QLoRA, multi-GPU, **the largest
open risk,
46     currently unquantified**) ? that stays explicitly **out of scope** for this budget. ?
**[03](03-PLATFORM-OPTIONS.md)**, **[06](06-RISKS-AND-OPEN-QUESTIONS.md)**.
47     4. **Docker is optional.** Serverless/Cloud Run paths need an image; a **DO Droplet** or
**SkyPilot**
48     path can run the repo in a plain `venv`/`conda` with no container. We recommend a
container **where you
49     go serverless* and a VM/SkyPilot path otherwise. ? **[03](03-PLATFORM-OPTIONS.md)**.
50     5. **Per-video billing falls out of a serverless per-invocation metering model + a cost
ledger on the
51     already-shipped `jobs` table.** ? **[05](05-COST-ATTRIBUTION.md)**.
52     6. **Cheapest-first: M0 costs $0** ? a CPU container/venv runs the **committed
synthetic, video-free
53     regression fixtures** ([`golden_fixtures.py`](../backend/eval/golden_fixtures.py))
in CI, no GPU,
54     no WSL. Only spend a dollar at M1. ? **[04](04-BUILD-AND-TEST-PLAN.md)**.
55     7. **The WSL detectors run seamlessly off the Windows box ? and we prove it on
DigitalOcean.** The
56     `DetectorRunner` ABC is already built for the swap (WSL plumbing is an isolated
mixin; the `Frame,
57     Visibility,X,Y` CSV is the parity contract); a DO GPU is Ubuntu+CUDA = your WSL conda
env, **minus
58     WSL**. A dedicated **M3.5 parity test on DO** is the guarantee, and it doubles as the
59     `DESKTOP_APP_PLAN` P0 spike. ? **[02](02-COMPUTE-MAP-AND-CONTRACT.md)**,
**[04](04-BUILD-AND-TEST-PLAN.md)**, **[06](06-RISKS-AND-OPEN-QUESTIONS.md)**.
60
61     ---
62
63     ## The budget tension you must decide (see [03](03-PLATFORM-OPTIONS.md) + [06](06-RISKS-
AND-OPEN-QUESTIONS.md))
64
65     You have **$200 in DigitalOcean**, and you're open to **GCS + a Cloud-Run-like**
architecture. These pull
66     in different directions for the **side goal** (exact per-video billing):
67
68     - **DigitalOcean** has the credits + **Spaces** (S3-compatible, **1 TiB egress
included** ? the best
69     video-download deal), but **no true serverless GPU**, so per-video metering is coarser
(you bill
70     wall-clock on a started/stopped GPU Droplet).
71     - **GCP Cloud Run (GPU) / RunPod Serverless** give **scale-to-zero + exact per-

```

```

invocation GPU-seconds**
72      ? which *is* your per-video bill ? but the $200 isn't DO credit there (GCP's
$300/90-day trial is the
73      clean free tier; RunPod's "free credit" is small).
74
75      **Recommended split:** prototype M0?M2 on **DO (use the credits) + DO Spaces**; stand up
the M3 inference
76      *endpoint* where per-invocation metering is native (Cloud Run GPU / RunPod Serverless).
Final call is
77      owner's ? tracked in [06](06-RISKS-AND-OPEN-QUESTIONS.md).
78
79      ---
80
81      ## Contents
82
83      | # | Doc | What it answers |
84      |---|---|---|
85      | **00** | [**HANDOFF ? resume here**](HANDOFF.md) | **? Current status (CPU e2e
working) + the next build (M3.5 native WASB). Read first.** |
86      | 01 | [Existing plans & the gap](01-EXISTING-PLANS-AND-GAP.md) | What we've already
planned for decoupling, and precisely what's unbuilt. |
87      | 02 | [Compute map & data contract](02-COMPUTE-MAP-AND-CONTRACT.md) | Where
GPU/CPU/WSL/API actually live in the 3 repos; what must move to a bucket. |
88      | 03 | [Platform options](03-PLATFORM-OPTIONS.md) | DO vs GCP vs serverless GPU; Docker-
optional alternatives; provider matrix. |
89      | 04 | [Build & test plan](04-BUILD-AND-TEST-PLAN.md) | The two entrypoints, bucket
layout, the M0?M4 milestone ladder, first PR. |
90      | 05 | [Cost attribution](05-COST-ATTRIBUTION.md) | Per-video inference billing + per-
run training cost segmentation for any user. |
91      | 06 | [Risks & open questions](06-RISKS-AND-OPEN-QUESTIONS.md) | The legal/scope/budget
gates and the decisions only the owner can make. |
92      | 07 | [Compute abstraction](07-COMPUTE-ABSTRACTION.md) | How we segment GPU vs CPU work
+ a spec-aware `DeviceContext`/`DeviceProfile` layer (the "GPU Runner"). |
93
94      ---
95
96      ## Verification status (checked against code 2026-06-20)
97
98      These load-bearing facts were confirmed against the current tree (not just the older
strategy docs):
99
100     - ? **Async-job control plane shipped** ? `jobs` table at
[database.py:188](../../backend/infrastructure/database.py), worker at
[api/job_worker.py](../../backend/api/job_worker.py), `api/process` ? 202 + `job_id`.
101     - ? **`jobs` table has no cost/tenant columns yet** (cols: id, video_path, segmenter,
player_pool, max_frames, status, policy, result, progress, timings) ? so the cost-ledger delta
in [05](05-COST-ATTRIBUTION.md) is net-new.
102     - ? **Gen-0 harness exists** ?
[training/gen0/harness.py](../../training/gen0/harness.py) (one-command train?eval?gate).
103     - ? **Committed synthetic, video-free regression fixtures exist** ?
[backend/eval/golden_fixtures.py](../../backend/eval/golden_fixtures.py) ? M0 is a genuine $0
CI gate.
104     - ? **`output` path still hardcoded** ?
[trajectory_hybrid.py:196,272](../../backend/pipeline/segmenters/trajectory_hybrid.py) (a real
bucket-relocation blocker).
105     - ? **No Dockerfile anywhere** in any of the three repos ? the container is greenfield.
106     - ? **Collector keys by human `video_id`, not MD5** ? `curate.py`; the export manifest
omits an `md5` field, which the MD5-keyed consumer needs (see [02](02-COMPUTE-MAP-AND-
CONTRACT.md)).
107
108     ## Provenance
109
110     Built from a 13-agent research workflow (?970k tokens, 5 plan-harvest + 3 compute-map +
2 cloud-provider
111     agents, then synthesis + an adversarial completeness critic), cross-checked against the

```

```

owner's strategy
112 docs and the live code. Cost/price figures are **approximate ? verify current provider
terms before
113 committing budget.**
114

```

7. user (2026-06-20T08:20:53.755Z)

```

1      # 02 . Compute map & data contract
2
3      **Status:** research synthesis, code-verified 2026-06-20 . **Date:** 2026-06-20
4
5      > What actually consumes GPU/CPU/WSL/external-API across the three repos, ranked by how
hard each is to
6      > lift into a Linux cloud-GPU container ? plus the one data contract everything keys on.
7
8      ## 1. Compute footprint of `badminton-highlight-indexer` (the consumer/trainer)
9
10     | Component | Needs | Portability risk ? cloud Linux GPU |
11     |---|---|---|
12     | **`gemini` segmenter** (default) | external paid API (`GEMINI_API_KEY`) | **LOW.**
Pure HTTP; no GPU. Silently returns **zero** segments without the key ? inject as a secret. |
13     | **`motion` segmenter** | CPU (OpenCV frame-diff) | **NONE.** Pure CPU, no API key. |
14     | **`yolo_hybrid` segmenter** | GPU (torchvision Faster R-CNN + ByteTrack); auto CPU-
fallback | **LOW.** Pure-pip; runs on CPU or CUDA. Pin `supervision<0.30` (ByteTrack API
drifts). Pre-bake COCO weights into the image to avoid first-run hub pulls. |
15     | **`trajectory_hybrid` / TrackNet / WASB** | **GPU + WSL2 bridge** (TensorFlow);
hardcoded `~/models` paths, hardcoded `device='cuda'` (no CPU fallback); **un-vendored**
research repos; TrackNet healthcheck requires a visible GPU | **HIGH ? but the swap seam is
already built.** Runs *only* via `wsl -d Ubuntu ? conda activate` **on Windows** ? on a Linux
GPU box WSL is **dropped, not ported** (the conda env runs natively). The `DetectorRunner` ABC
([`detectors/base.py`](../../backend/pipeline/detectors/base.py)) exists *expressly* to de-risk
this: WSL plumbing is a separate `WslCommandMixin` (a native runner must not inherit it),
Windows behavior is **100% preserved,** and the **parity contract is the `Frame,Visibility,X,Y`
CSV** ? any runner emitting it reuses `parse_trajectory_csv` + `trajectory_to_action_windows`
verbatim, so everything downstream is byte-identical. The `LinuxNativeRunner` impl doesn't exist
yet / has never been run ? prove it via the **M3.5 parity test on DigitalOcean** ([04](04-BUILD-
AND-TEST-PLAN.md)). |
16     | **`fusion_hybrid` / Gen-0 head** | CPU/tiny GPU (numpy/scipy) | **NONE.**
[`training/gen0/harness.py`](../../training/gen0/harness.py): ~1M params, ~4.5 GB VRAM, trains
in *minutes*. This is the cheap retrain target. |
17     | **`ffmpeg` chunking / stitching** | CPU (`libx264`) | **LOW.** No nvenc/driver-codec
dependency (a portability win) ? `apt-get install ffmpeg` is enough. |
18     | **Gen-2 VLM** (ms-swift / QLoRA Qwen3-VL) | **multi-GPU** (6?8-GPU GRPO reference; 24
GB VRAM floor) | **OUT OF SCOPE for $200.** **"Largest open risk, currently unquantified." Do
not attempt on this budget. |
19
20     **Two structural container blockers beyond WSL:**
21     - **TF-vs-torch runtime conflict.** TrackNet is TensorFlow; the rest is PyTorch. Their
pip-bundled CUDA
22     userspaces disagree ? "works alone, segfaults together." Fix = **two single-
framework images** (a torch
23     image, a TF image) sharing a `/scratch` volume. So the container topology itself is
non-trivial.
24     - **CUDA wheel selection.** `torch>=2.12.0` pinned with no index ? the
Dockerfile/installer must pick
25     `--index-url ?/cu124` vs `?/cpu` explicitly.
26
27     **Path / I/O blockers (confirmed live):**
28     - Hardcoded literal `"output"` in
[`trajectory_hybrid.py:196,272`](../../backend/pipeline/segmenters/trajectory_hybrid.py) ? not
redirected by `--output-dir`.
29     - Local-file assumptions: `cv2.VideoCapture`, `ffmpeg -i <path>`, whole-file MD5,
`allowed_video_root`
30     check in [`main.py`](../../backend/main.py). No URL/stream path ? needs a

```

```

`fetch(uri)?local` + `put` shim.
31
32     ## 2. Compute footprint of `sports-data-collector` (the producer/curator)
33
34     CPU-first; "the gap is narrow." Only GPU touchpoint is **opt-in `nvenc`** (--encoder
nvenc`, commit #28)
35     for proxy transcode + batch transcode ? defaults to CPU `libx264`. Shares the same
local-filesystem + MD5
36     assumptions. The export **`manifest.json` omits an `md5` field** (`proxy.py` carries
only `source_md5` /
37     `proxy_md5`); the collector keys by **human `video_id`** (`curate.py`), while the
indexer keys by **file
38     MD5** ? see §4.
39
40     ## 3. Compute footprint of `rally-annotator` (the labeler)
41
42     Effectively **no heavy compute** ? a VLC 3.x Lua extension a human drives to mark rally
`start`/`end` +
43     ending reason ? <video>.rallies.csv. Nothing to containerize; it's the *source* of the
labels that must
44     land in the bucket.
45
46     ## 4. The one data contract everything keys on
47
48     ```
49     rally-annotator           sports-data-collector           badminton-highlight-indexer
50     (label by hand) ????????? (curate + license + export) ???
(validate?segment?index?stitch?eval?train)
51     <video>.rallies.csv       SQLite ? manifest.json + CSVs     keyed by file MD5
52     ```
53
54     - **Identity:** the indexer keys videos by **whole-file MD5** (`compute_video_id`, 64
KB-chunked); the
55     collector keys by **human `video_id`**. The manifest's `local_path` ties them
together. **Any re-encode
56     changes the MD5** and silently breaks the join.
57     - **Intervals:** temporal `[start, end)` in **decimal seconds**; ending-reason taxonomy
58     (winner/forced_error/unforced_error/service_fault/let/other) is the cross-sport label
set.
59     - **Train/serve wall (ADR-008):** the boundary between training and serving is an
**artifact bundle**
60     (weights + manifest), **never a code import** ? keyed by content/semver.
61
62     ### What this means for a bucket workflow
63
64     1. **Add `md5` to the collector export manifest** (or decide *original-vs-proxy* keying)
**before** feeding
65     a bucket ? otherwise the MD5-keyed consumer can't resolve labels?video. *This is an
M2 prerequisite, not
66     a footnote.*
67     2. **Store byte-identical files** the labels were made against. If you must proxy-
transcode for size, carry
68     an explicit `original_md5` ? `proxy_md5` mapping so the join survives.
69     3. A cloud `train`/`infer` job's **inputs** = `{video object(s), label CSV(s),
manifest.json}` from the
70     bucket; its **outputs** = `{versioned weight bundle}` (train) or `{highlights.mp4,
intervals.csv,
71     report.json}` keyed by `video_id` (infer). These map cleanly onto the existing per-
video output layout.
72
73     ### Inputs/outputs a cloud job must exchange (summary)
74
75     | Verb | Pulls from bucket | Pushes to bucket |
76     |---|---|---|
77     | `train` | `videos/`, `labels/`, `manifest.json` (+ latest checkpoint to resume) |

```

```
`weights/<semver>/{model, manifest}` + periodic `weights/checkpoints/` |
78 | `infer` | one `videos/<md5>.mp4` | `outputs/<video_id>/{highlights.mp4, intervals.csv,
report.json}` |
79
```

8. user (2026-06-20T08:20:54.168Z)

```
1 # Setting up a new machine ? zero ? training & inference
2
3 **The canonical "spin up a box and make the models ready" runbook.** Copy-pasteable;
followable by
4 any human. DigitalOcean is the worked example (GCP/AWS/Paperspace are analogous ? only
the provision
5 step differs).
6
7 > **Last verified:** 2026-06-20 on a DigitalOcean Droplet (Ubuntu 24.04, 2 vCPU / 3.8
GB, **no GPU**).
8 > Bootstrap succeeded (torch 2.12.1+cpu); full mocked suite **922 passed / 11 skipped**;
a **secret-free**
9 > CPU inference (no GPU, no API key) detected **2 rallies** and persisted them to
SQLite.
10 >
11 > ? Assumes the DECOUPLED_COMPUTE PRs are merged: the CPU-mock detector seam,
`deploy/digitalocean/bootstrap.sh`,
12 > the storage layer, and the worker dispatcher (#251). The **bucket** steps (§7) are
config-driven and
13 > **verified against real DO Spaces + GCS** ? see the appendix.
14
15 ## What you'll have at the end
16
17 A machine that can: run the test suite (green), run **secret-free CPU inference**
(rallies, no GPU/keys),
18 run **real Gemini inference** (with a key), and run **Gen-0 model training** ? plus
pointers to the GPU /
19 native-detector path and bucket-driven train/infer.
20
21 ---
22
23 ## 1. Credentials to create once (least-privilege)
24
25 | # | Credential | Where | Needed for |
26 |---|---|---|---|
27 | 1 | **SSH keypair** (public key added to the provider) | `ssh-keygen -t ed25519 -f
~/.ssh/rally` ? paste `.pub` into DO ? Settings ? Security | logging into the box |
28 | 2 | **DigitalOcean API token** (read+write) | DO ? API ? Tokens | `doctl` provisioning
(optional ? you can click-create) |
29 | 3 | **GitHub repo read access** | a read-only **deploy key** *or* a fine-grained
**PAT** (`Contents: Read`) | `git clone` on the box (or use the `scp` path in §3) |
30 | 4 | **`GEMINI_API_KEY`** | Google AI Studio / Vertex | **real** Gemini inference only
(§6b). The secret-free path (§6a) needs none. **Rotate before sharing.** |
31 | 5 | **Bucket creds** ? DO Spaces key/secret, or GCS SA JSON, or Drive SA JSON + folder
| provider console | bucket-driven train/infer (§7, *pending PR-B*) |
32 | 6 | **GPU box** (Paperspace free M4000 / GPU Droplet) | provider console | real
WASB/TrackNet detectors + faster runs (§8) |
33
34 **Secret hygiene:** never put a secret in the repo, a PR, or `config.json`. Set them as
env vars on the box
35 (or a git-ignored `.env`). Scope each as narrowly as the UI allows and rotate/revoke
when done.
36
37 ---
38
39 ## 2. Provision the machine
40
41 - **CPU box** (suite + secret-free stub inference + Gen-0 training plumbing): cheapest
```



```

**Ubuntu 24.04**
42     Droplet, **2 vCPU / 4 GB**, region nearest you, **attach the SSH key** (#1). ~$0.06/hr
? destroy when done.
43     - **GPU box** (real WASB/TrackNet detectors ? real trajectory extraction + training): a
**DO GPU Droplet**.
44     ? You **cannot add a GPU to a CPU Droplet** ? it's a separate machine type, in **GPU
regions**
45     (`nyc2`/`tor1`/`atl1`, **not** `blr1`), with **dual local NVMe**. **RTX 4000 Ada (20
GB, ~$0.76/hr)** is
46     plenty for WASB + Gen-0; the `gpu-*-base` image preinstalls the NVIDIA driver. **Co-
locate the Space in the
47     GPU box's region** for cheap corpus-pull.
48
49     CLI provision (after `doctl auth init` with token #2 ? **verify size/image slugs first**
via `doctl compute size list`):
50     ```bash
51     # CPU box:
52     doctl compute droplet create rally-box --image ubuntu-24-04-x64 --size s-2vcpu-4gb \
53     --region blr1 --ssh-keys <your-key-fp> --wait
54     # GPU box (RTX 4000 Ada):
55     doctl compute droplet create rally-gpu --image gpu-h100x1-base --size gpu-4000adax1-20gb
\
56     --region tor1 --ssh-keys <your-key-fp> --wait
57     doctl compute droplet list # note the IP -> ssh -i ~/.ssh/rally root@<IP>
58     ```
59
60     ---
61
62     ## 3. Get the code onto the box
63
64     **Option A ? git clone** (private repo, so authenticate with #3):
65     ```bash
66     ssh -i ~/.ssh/rally root@<IP>
67     git clone git@github.com:avidullu/badminton-highlight-indexer.git ~/rally # via deploy
key
68     # or: git clone https://<PAT>@github.com/avidullu/badminton-highlight-indexer.git
~/rally
69     cd ~/rally
70     ```
71
72     **Option B ? push a local checkout** (no GitHub access needed; what the maintainer
used):
73     ```bash
74     # on your machine, from the repo root:
75     git archive --format=tar.gz -o /tmp/rally.tgz HEAD
76     scp -i ~/.ssh/rally /tmp/rally.tgz root@<IP>:/root/
77     ssh -i ~/.ssh/rally root@<IP> 'mkdir -p ~/rally && tar -xzf ~/rally.tgz -C ~/rally'
78     ```
79
80     ---
81
82     ## 4. Bootstrap (one command)
83
84     ```bash
85     cd ~/rally
86     ./deploy/digitalocean/bootstrap.sh # CPU box
87     # or
88     ./deploy/digitalocean/bootstrap.sh --gpu # GPU box (installs CUDA torch)
89     ```
90
91     It mirrors CI: installs `ffmpeg` + opencv runtime, creates `.venv`, installs CPU-or-CUDA
`torch`, then
92     `pip install -e .[dev]`, and prints a device check (`cuda_available True/False`).
93
94     <details><summary>Manual equivalent (if you can't run the script)</summary>

```

```

95
96     ```bash
97     sudo apt-get update
98     sudo apt-get install -y python3 python3-venv python3-pip git ffmpeg libgl1
99     sudo apt-get install -y libglib2.0-0t64 || sudo apt-get install -y libglib2.0-0
100    python3 -m venv .venv && . .venv/bin/activate && python -m pip install -U pip
101    pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu # or
.../whl/cu124 on GPU
102    pip install -e .[dev]
103    ```
104    </details>
105
106    ---
107
108    ## 5. Verify the machine (the smoke gate)
109
110    ```bash
111    . .venv/bin/activate
112    python -m pytest -q # expect all green (922 passed / 11 skipped on the current
tree)
113    ```
114    A green suite proves the whole pipeline runs on this box with **no GPU, no WSL, no
bucket, no API key**.
115
116    ---
117
118    ## 6. Run inference
119
120    ### 6a. Secret-free CPU inference (no GPU, no API key) ? the "mock a GPU with a CPU"
path
121
122    Drop a permissive smoke `config.json` in the repo root (the CLI reads `./config.json`
from the cwd):
123    ```bash
124    cd ~/rally && cat > config.json <<'JSON'
125    {
126        "sport": "badminton",
127        "indexing": { "default_segmenter": "wasb_hybrid", "detector_impl": "stub",
"skip_ai_handoff": true },
128        "validation": { "min_fps": 0, "min_blur_threshold": 0, "max_scene_cuts_per_minute":
1000,
129                        "relevance": { "court_pixels_min_ratio": 0.0 } }
130    }
131    JSON
132
133    # a throwaway 12s 1280x720 test clip
134    ffmpeg -y -f lavfi -i testsrc=duration=12:size=1280x720:rate=30 -pix_fmt yuv420p
/root/sample.mp4
135
136    # run ? env -u drops any Gemini key to prove none is needed
137    env -u GEMINI_API_KEY RALLY_DETECTOR_IMPL=stub \
138    indexer --video-path /root/sample.mp4 --segmenter wasb_hybrid --output-dir ~/rally/out
139    ```
140    Expect: `detector_impl=stub ? CPU mock runner`, `Phase 3 SKIPPED (fully-local)`, and
141    `Indexed 2 play segments` in `~/rally/out/sports_indexer.db`. *(`detector_impl=stub`
swaps the GPU+WSL
142    detector for a deterministic CPU one; `skip_ai_handoff=true` makes candidate windows the
rallies, so no
143    Gemini handoff happens.)*
144
145    ### 6b. Real inference (Gemini segmenter)
146
147    ```bash
148    export GEMINI_API_KEY="" # secret stays in the env, never in
the repo

```

```

149 # use a real badminton clip (must pass validation: ?1280x720, ?30fps, a green court,
etc.)
150 indexer --video-path /root/match.mp4 --segmenter gemini --output-dir ~/rally/out
151 # or the hybrid: --segmenter wasb_hybrid (WASB needs a GPU box, S8) with
skip_ai_handoff=false
152 ```
153
154 ---
155
156 ## 7. Object storage ? bucket-driven train/infer (S3-compatible / GCS / Drive)
157
158 The worker (`python -m backend.worker`) reads/writes by **URI**, so you choose a backend
with **config +
159 credentials ? no code change**. A bare path / `local://` is a passthrough (today's
behaviour); `s3://` and
160 `gcs://` go to a bucket. Install only the extra you use (cloud SDKs are lazy-imported).
161
162 > **Verified end-to-end 2026-06-20:** the worker bucket round-trip (pull video ? infer ?
push outputs) passed
163 > against **real DigitalOcean Spaces** (sgp1), a real **GCS** client (emulator), and
**MinIO**.
164
165 ### 7a. S3-compatible ? AWS S3 / DigitalOcean Spaces / Cloudflare R2 / MinIO
166
167 One backend for all four ? they differ only by `endpoint_url` + region. `pip install -e
.[storage-s3]`.
168
169 **Credentials** ? `~/.aws/credentials` (boto3's default chain; **never** in the repo):
170 ```ini
171 [default]
172 aws_access_key_id = <key>
173 aws_secret_access_key = <secret>
174 ```
175 **Config** ? `config.json` `storage.s3`:
176 ```jsonc
177 "storage": { "backend": "local", "s3": {
178   "endpoint_url": "https://sgp1.digitaloceanspaces.com", // DO Spaces (VERIFIED);
region = the Space's, e.g. sgp1
179   "region": "sgp1"
180   // AWS S3 : omit endpoint_url; set region (e.g. ap-south-1)
181   // R2 : endpoint_url https://<account>.r2.cloudflarestorage.com ; region "auto"
182   // MinIO : endpoint_url http://host:9000 ; region us-east-1 ; "addressing_style":
"path"
183 }}
184 ```
185 **Run** (secret-free flavor; use `--segmenter gemini` for real detection):
186 ```bash
187 env -u GEMINI_API_KEY RALLY_DETECTOR_IMPL=stub python -m backend.worker infer \
188   --video-uri s3://<bucket>/videos/clip.mp4 --out-uri s3://<bucket> \
189   --segmenter wasb_hybrid --set indexing.skip_ai_handoff=true
190 ```
191
192 ### 7b. Google Cloud Storage
193
194 `pip install -e .[storage-gcs]`. **Credentials** ? a service-account JSON (SA with
**Storage Object Admin**):
195 ```bash
196 export GOOGLE_APPLICATION_CREDENTIALS=/path/to/sa.json
197 ```
198 **Config** ? `storage.gcs.project` (no endpoint ? default `googleapis.com`):
199 ```jsonc
200 "storage": { "backend": "local", "gcs": { "project": "<gcp-project-id>" } }
201 ```
202 Same `worker infer` command with `gcs://<bucket>/?` URIs. *(On-box test with no GCP
creds:

```

```

203     `pip install gcp-storage-emulator && gcp-storage-emulator start --port 9023 --in-memory`
+
204     `export STORAGE_EMULATOR_HOST=http://127.0.0.1:9023`.)*)
205
206     ### 7c. Google Drive *(corpus source ? follow-up backend)*
207
208     `gdrive://` is recognised; the backend is a focused follow-up (for the read-only golden
corpus).
209
210     **Bucket layout:** `videos/ labels/ manifest.json weights/ outputs/<video_id>/` (see
`docs/DECOUPLED_COMPUTE/04`).
211
212     ---
213
214     ## 8. Run training (Gen-0 rally head)
215
216     CPU-runnable, minutes ? but it needs a **labeled corpus** (videos + `.rallies.csv`
labels + a manifest),
217     which a fresh box doesn't have. Bring it via $7's bucket (`worker train`, pending) or
`scp` it up, then:
218     ```bash
219     python -m training.gen0.harness --manifest <manifest.json> --floor <baseline> --version
0.1.0
220     ```
221     The harness runs train ? eval ? ship-gate and emits a versioned weight bundle. *(The
Gen-2 VLM is a
222     separate, GPU-heavy, separately-budgeted effort ? out of scope here.)*
223
224     ---
225
226     ## 9. GPU box ? port procedure (NVMe + native WASB)
227
228     The GPU box is **cattle, not a pet**: durable state lives in the bucket + git, so a
fresh box is running in
229     minutes. Port = provision ($2) ? code on ($3) ? NVMe ? `bootstrap.sh --gpu` ?
`gpu_setup.sh` ? pull corpus.
230
231     ```bash
232     # on the box, after $3 put the code at ~/rally:
233     cd ~/rally
234     sudo bash deploy/digitalocean/mount_scratch.sh && export TMPDIR=/scratch #
detect+mount the spare NVMe
235     ./deploy/digitalocean/bootstrap.sh --gpu # CUDA
torch; expects cuda_available True
236     . .venv/bin/activate && pip install -e .[storage-s3]
237     bash deploy/digitalocean/gpu_setup.sh # native
WASB conda env (no WSL) ? M3.5 enabler
238     # secrets (the only two non-reproducible items): Gemini key + bucket creds
239     mkdir -p /root/.keys && printf '%s' '<rotated-key>' > /root/.keys/GEMINI_API_KEY &&
chmod 600 /root/.keys/GEMINI_API_KEY
240     install -m600 /dev/stdin ~/.aws/credentials <<'INI'
241     [default]
242     aws_access_key_id = <key>
243     aws_secret_access_key = <secret>
244     INI
245     ```
246
247     - **NVMe scratch** (`mount_scratch.sh`): DO GPU Droplets ship a second raw NVMe; the
script formats+mounts it
248     at `/scratch` and the pipeline's sync-first decode runs there (`export
TMPDIR=/scratch` or `--scratch /scratch`).
249     Ephemeral, not in fstab. On a CPU box it just makes `/scratch` on the boot disk.
250     - **Native WASB** (`gpu_setup.sh`): replicates `docs/TRACKNET_WSL_SETUP.md`'s conda env
**natively on Linux**
251     (no WSL) ? the M3.5 enabler. ? Needs `wasb_badminton_best.pth.tar` present and the

```

```

**WASB-C3** weight
252     provenance cleared before sharing any derived model.
253     - **The detector itself:** real `--segmenter wasb_hybrid` extraction needs the Linux-
native `DetectorRunner`
254     (tracked M3.5, `docs/DECOUPLED_COMPUTE/04`). Until it lands, validate the train/infer
plumbing with
255     `detector_impl=stub`.
256     - **Teardown** = `doctl compute droplet delete rally-gpu`. Re-spin tomorrow identically.
257
258     ---
259
260     ## 10. Teardown
261
262     ```bash
263     rm -f ~/rally/config.json ~/sample.mp4          # remove the throwaway smoke artifacts
264     doctl compute droplet delete rally-box          # or destroy in the console
265     ```
266     Revoke any temporary token/key (#2?#5) and delete the SSH key from the provider when the
box is gone.
267
268     ---
269
270     ## Appendix ? verification log
271
272     | Step | Command | Result (2026-06-20, D0 Ubuntu 24.04, 2 vCPU, no GPU) |
273     |---|---|---|
274     | Bootstrap | `bootstrap.sh` | ? torch 2.12.1+cpu, deps + editable build |
275     | Smoke | `pytest -q` | ? 922 passed, 11 skipped (49s) |
276     | Secret-free infer (local) | $6a | ? 2 rallies, persisted to SQLite, `GEMINI_API_KEY`
unset |
277     | **S3 worker round-trip ? real D0 Spaces (sgp1)** | $7a | ? pulled
`s3://rally-e2e/videos/?` ? 2 rallies ? pushed `outputs/<md5>/intervals.csv` |
278     | **GCS worker round-trip** (emulator) | $7b | ? pulled `gcs://?` ? 2 rallies ? pushed
`gcs://?/outputs/<md5>/?` |
279     | S3 backend vs MinIO (put/get/list/stat + presigned GET) | $7a | ? |
280

```

9. user (2026-06-20T08:20:54.641Z)

```

1     # TrackNet / WASB on WSL2 ? setup & usage
2
3     Self-hosted shuttle-trajectory detectors run inside WSL2 (Ubuntu) and are bridged
4     to the Windows indexer via a subprocess adapter. See
`docs/archives/decisions/DECISIONS.md` (ADR-001)
5     for the why.
6
7     ## What's installed (WSL2 / Ubuntu 24.04)
8
9     | Thing | Location (WSL) |
10    |-----|-----|
11    | Miniconda | `~/miniconda3` |
12    | TrackNetV4 repo | `~/models/TrackNetV4` (https://github.com/TrackNetV4/TrackNetV4) |
13    | TrackNetV4 conda env | `TrackNetV4` (Python 3.9, TensorFlow 2.17 + bundled CUDA 12.3)
|
14    | WASB repo | `~/models/WASB-SBDT` (https://github.com/nttcom/WASB-SBDT) |
15    | WASB conda env | `wasb` (Python 3.8, torch 1.11.0+cu113) ? verified
`torch.cuda.is_available()==True` |
16    | Staging dir for videos | `~/clips` (avoids slow `/mnt/c` per-frame reads) |
17
18    GPU: RTX 2070 Super visible via `nvidia-smi` inside WSL (CUDA-on-WSL working).
19
20    Activate an env with:
21    ```bash
22    source ~/miniconda3/etc/profile.d/conda.sh
23    conda activate TrackNetV4    # or: conda activate wasb

```

```

24     ``
25
26     ## ?? Two upstream issues you must resolve before TrackNetV4 inference
27
28     1. **No pretrained weights are hosted.** The "Download" links in
29         `~/models/TrackNetV4/docs/RESULT.md` are placeholders `(#{})`. You will obtain
30         weights by **training** (`src/train.py`) on your annotated data ? which is the
31         plan. Until you have a `.h5`/`.keras` weights file, the segmenter's healthcheck
32         passes but `run_predict` will log an error that `weights_path` is unset.
33
34     2. **`predict.py` has a custom-objects bug.** `src/predict.py` references layers in
35         `custom_objects` that it never imports (`MotionIncorporationLayerV1`,
36         `MotionIncorporationLayerV2`, `CombineOutputs`, `MotionFramesInput`) ? it will
37         raise `NameError` on `load_model`. The imported layers are
38         `MotionPromptLayer`, `FusionLayerTypeA`, `FusionLayerTypeB`. Patch the
39         `custom_objects` dict in `main()` to match the layers your trained model
40         actually uses, e.g.:
41         ```python
42         model = load_model(model_weights_path, custom_objects={
43             'MotionPromptLayer': MotionPromptLayer,
44             'FusionLayerTypeA': FusionLayerTypeA, # if you trained TrackNetV4_TypeA
45             'FusionLayerTypeB': FusionLayerTypeB, # if you trained TrackNetV4_TypeB
46             'custom_loss': custom_loss,
47         })
48         ```
49
50     ## Wiring it into the indexer
51
52     The segmenter is `tracknet_hybrid` (`backend/pipeline/segmenters/tracknet_hybrid.py`),
53     backed by
54     `backend/pipeline/detectors/tracknet_runner.py`. Configure it in `config.json` under
55     `indexing.tracknet`:
56
57     ```jsonc
58     "tracknet": {
59         "wsl_distro": "Ubuntu",
60         "repo_dir": "~/models/TrackNetV4",
61         "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
62         "conda_env": "TrackNetV4",
63         "weights_path": "", // <-- set to the WSL path of your trained weights
64         "queue_length": 5,
65         "stage_in_wsl": true, // copy the video into ~/clips before inference
66         "wsl_stage_dir": "~/clips",
67         "timeout_sec": 1800, // default 30 min; 0 = no timeout
68         "velocity_threshold": 0.02 // normalised shuttle displacement/sec to count as "in
69         flight"
70     }
71     ```
72
73     Then select it as the segmenter (e.g. set `indexing.default_segmenter` to
74     `tracknet_hybrid`, or pass it through whatever segmenter-selection your run uses).
75
76     ### Pipeline shape (mirrors `yolo_hybrid`)
77
78     1. **Phase 1 ? healthcheck:** activates the WSL env, confirms TF imports and a GPU
79         is visible. Fails fast with a clear message otherwise.
80
81     2. **Phase 2 ? TrackNet:** runs `predict.py` on the whole video, parses the
82         `Frame,Visibility,X,Y` CSV, and converts the shuttle trajectory into candidate
83         rally windows (visible + moving above `velocity_threshold`, grouped by
84         `dead_time_merge_gap`). Raw candidates are persisted for fine-tuning (same table
85         as YOLO candidates).
86
87     3. **Phase 3 ? AI handoff:** each candidate window is sent to the configured AI
88         provider for semantic boundary confirmation (identical to `yolo_hybrid`).
89
90     4. **Phase 4 ? DB:** confirmed segments written and linked back to their candidate.

```

```

87     ### Quick manual smoke test (inside WSL, once you have weights)
88
89     ```bash
90     source ~/miniconda3/etc/profile.d/conda.sh && conda activate TrackNetV4
91     cd ~/models/TrackNetV4/src
92     cp /mnt/c/Users/avidu/Projects/badminton-highlight-indexer/<clip>.mp4 ~/clips/
93     python predict.py --video_path ~/clips/<clip>.mp4 \
94         --model_weights <your_weights> --output_dir ~/clips/out
95     head ~/clips/out/<clip>_predict.csv    # expect: Frame,Visibility,X,Y
96     ```
97
98     ## WASB (multi-sport ? and the faster path to a baseline)
99
100     **WASB ships pretrained weights**, including badminton, in
101     `~/models/WASB-SBDT/pretrained_weights/`:
102
103     - `wasb_badminton_best.pth.tar` (the WASB model)
104     - plus `tracknetv2_badminton_best.pth.tar`, `monotrack_badminton_best.pth.tar`,
105       `restracknetv2_badminton_best.pth.tar`, `ballseg_badminton_best.pth.tar`,
106       `deepball*_badminton_best.pth.tar` ? WASB re-implements all baselines.
107
108     This means **WASB can detect the shuttle on badminton video today**, whereas
109     TrackNetV4 has no hosted weights and must be trained first. If you want a working
110     trajectory baseline before your fine-tuning data is ready, WASB is the quicker
111     route.
112
113     Note: WASB's upstream repo documents dataset *evaluation*, not custom-video inference ?
114     so **this repo already ships the wrapper**: `backend/pipeline/detectors/wasb_infer.py`
115     (staged into WSL by `backend/pipeline/detectors/wasb_runner.py:WasbRunner`) runs the
116     detector + tracker on arbitrary video and emits the `Frame,Visibility,X,Y` CSV the
117     shared
118     windowing logic parses. The live `wasb_hybrid` segmenter uses it. Configure it via
119     `indexing.wasb` in `config.json`; see CODE_MAP §2.1/§2.2. (No need to build this ? it
    exists.)

```

10. assistant (2026-06-20T08:20:56.799Z)

Now let me read the deploy scripts to get exact paths and commands.

11. user (2026-06-20T08:20:58.637Z)

```

1     #!/usr/bin/env bash
2     #
3     # Bootstrap a DigitalOcean / Paperspace Ubuntu box to run the rally pipeline.
4     #
5     #   ./deploy/digitalocean/bootstrap.sh           # CPU box (Droplet)           ? M0 /
M1-cpu
6     #   ./deploy/digitalocean/bootstrap.sh --gpu     # GPU box (Paperspace / GPU Droplet) ?
M1 / M3.5
7     #
8     # Mirrors .github/workflows/ci.yml (Ubuntu) so the box matches CI exactly.
9     # STORAGE-FREE + SECRET-FREE: proves the pipeline runs on the box via the mocked
10    # test suite. No bucket, no Gemini key, no WSL required. See
11    ../../docs/DECOUPLED_COMPUTE/.
12    set -euo pipefail
13
14    GPU=0
15    [ "${1:-}" = "--gpu" ] && GPU=1
16
17    echo ">> [1/4] System libraries (ffmpeg + opencv runtime)"
18    sudo apt-get update
19    sudo apt-get install -y python3 python3-venv python3-pip git ffmpeg libgl1
20    # opencv needs libglib2.0-0; the package name differs across Ubuntu releases (t64 on
21    24.04).

```

```

20     sudo apt-get install -y libglib2.0-0t64 || sudo apt-get install -y libglib2.0-0
21
22     echo ">> [2/4] Python venv (.venv)"
23     python3 -m venv .venv
24     # shellcheck disable=SC1091
25     . .venv/bin/activate
26     python -m pip install --upgrade pip
27
28     echo ">> [3/4] torch (${[ "$GPU" = 1 ] && echo CUDA || echo CPU}) + package (editable,
dev tooling)"
29     if [ "$GPU" = 1 ]; then
30         # CUDA 12.4 wheels. NOTE: verify your GPU arch is supported by this build.
31         # The Paperspace free M4000 is Maxwell (sm_52) ? old archs are sometimes dropped;
32         # if `torch.cuda.is_available()` is False on an M4000, use a P4000/RTX4000 or pin
33         # an older torch. (docs/DECOUPLED_COMPUTE/03 + 06.)
34         pip install torch torchvision --index-url https://download.pytorch.org/whl/cu124
35     else
36         pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
37     fi
38     # Runtime deps + pytest/ruff/mypy. The 'reporting' extra (private obsreport over SSH)
39     # is intentionally omitted ? it is a soft dep and its tests skip when absent.
40     pip install -e .[dev]
41
42     echo ">> [4/4] Device check"
43     python - <<'PY'
44     import torch
45     ok = torch.cuda.is_available()
46     print("torch", torch.__version__, "| cuda_available", ok,
47           "| device", (torch.cuda.get_device_name(0) if ok else "CPU only"))
48     PY
49
50     cat <<'EOF'
51
52     >> Bootstrap complete. Run the storage-free, secret-free smoke (mocked suite):
53
54     . .venv/bin/activate && python -m pytest -q
55
56     Expect ~900 passed. That proves the whole pipeline runs on this box with no GPU,
57     no WSL, no bucket and no API key. (On a GPU box the device check above should
58     print cuda_available True.)
59 EOF
60

```

12. user (2026-06-20T08:20:59.985Z)

```

1     #!/usr/bin/env bash
2     #
3     # Native (no-WSL) WASB shuttle-detector setup for a Linux GPU box ? the M3.5 enabler for
4     # real trajectory extraction. Replicates docs/TRACKNET_WSL_SETUP.md's WSL conda env
5     # *natively* on Linux. Run AFTER ./bootstrap.sh --gpu (which provides CUDA torch for the
6     # main venv); this builds a SEPARATE conda env for WASB (torch 1.11) ? never co-
installed
7     # with the indexer's torch (framework conflict).
8     #
9     # Requires an NVIDIA GPU + driver (the D0 `gpu-*-base` image ships it).
10    #
11    # ? WASB-C3: the pretrained badminton weights' license / source-dataset provenance is
NOT
12    # verified here. Fine for internal extraction + Gen-0 bootstrapping; clear it before
sharing
13    # any derived trajectories or model. See docs/DECOUPLED_COMPUTE/06.
14    set -euo pipefail
15
16    MODELS_DIR="${RALLY_MODELS_DIR:-$HOME/models}"
17    CONDA_DIR="${CONDA_DIR:-$HOME/miniconda3}"

```



```

18 WASB_REPO="$MODELS_DIR/WASB-SBDT"
19 mkdir -p "$MODELS_DIR"
20
21 echo "[gpu] [1/5] miniconda (native Linux)"
22 if [ ! -x "$CONDA_DIR/bin/conda" ]; then
23     curl -sSL https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -o
/tmp/mc.sh
24     bash /tmp/mc.sh -b -p "$CONDA_DIR" && rm -f /tmp/mc.sh
25 fi
26 # shellcheck disable=SC1091
27 source "$CONDA_DIR/etc/profile.d/conda.sh"
28
29 echo "[gpu] [2/5] WASB-SBDT repo"
30 if [ ! -d "$WASB_REPO/.git" ]; then
31     git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO"
32 fi
33
34 echo "[gpu] [3/5] conda env 'wasb' (py3.8 + torch 1.11.0+cu113 ? the verified WASB
stack)"
35 conda env list | grep -q '/envs/wasb$' || conda create -y -q -n wasb python=3.8
36 conda activate wasb
37 python -m pip install -q torch==1.11.0+cu113 torchvision==0.12.0+cu113 \
38     --extra-index-url https://download.pytorch.org/whl/cu113
39 [ -f "$WASB_REPO/requirements.txt" ] && python -m pip install -q -r
"$WASB_REPO/requirements.txt" || true
40
41 echo "[gpu] [4/5] pretrained badminton weights"
42 WEIGHTS="$WASB_REPO/pretrained_weights/wasb_badminton_best.pth.tar"
43 if [ -f "$WEIGHTS" ]; then
44     echo "[gpu]         found $WEIGHTS"
45 else
46     echo "[gpu]         !! MISSING $WEIGHTS ? fetch wasb_badminton_best.pth.tar
(Drive/bucket) before extraction."
47 fi
48
49 echo "[gpu] [5/5] GPU sanity check (in the wasb env)"
50 python - <<'PY'
51 import torch
52 ok = torch.cuda.is_available()
53 print("[gpu]         torch", torch.__version__, "| cuda_available", ok,
54       "| device", (torch.cuda.get_device_name(0) if ok else "CPU"))
55 assert ok, "CUDA not available in the wasb env ? check the GPU driver / base image"
56 PY
57
58 echo "[gpu] native WASB env ready.  Activate with:  source
$CONDA_DIR/etc/profile.d/conda.sh && conda activate wasb"
59 echo "[gpu] (the LinuxNativeRunner ? backend/pipeline/detectors ? drives extraction once
it lands; M3.5.)"
60

```

13. user (2026-06-20T08:21:00.068Z)

```

1  #!/usr/bin/env bash
2  #
3  # Detect a spare local NVMe and mount it at /scratch for fast video decode.
4  #
5  # DO GPU Droplets ship a SECOND raw NVMe (the "scratch" disk), unmounted by default; a
6  # regular CPU box has only the boot disk ? then we just create /scratch on it. The hot
path
7  # is sync-first (copy video local -> ffmpeg-decode there), so scratch belongs on the
fast
8  # local disk. Point the pipeline at it with  export TMPDIR=/scratch  or  --scratch
/scratch.
9  #
10 # EPHEMERAL by design (wiped on destroy / may not survive a power-cycle) and

```

```

deliberately NOT
11     # added to /etc/fstab (a renamed device could block boot). Nothing durable lives here.
12     set -euo pipefail
13     MNT=/scratch
14     mkdir -p "$MNT"
15
16     root_src=$(findmnt -n -o SOURCE / 2>/dev/null || true)
17     root_disk=$(lsblk -no PKNAME "$root_src" 2>/dev/null | head -1 || true)
18     [ -z "$root_disk" ] && root_disk=$(basename "${root_src:-}" | sed -E 's/p?[0-9]+$/ /')
19
20     cand=""
21     while read -r name type; do
22         [ "$type" = "disk" ] || continue
23         [ "$name" = "$root_disk" ] && continue
24         bytes=$(lsblk -bdno SIZE "/dev/$name" 2>/dev/null | head -1)
25         [ "${bytes:-0}" -lt 10000000000 ] && continue # skip tiny config drives
26         if lsblk -no MOUNTPOINT "/dev/$name" | grep -q '/'; then continue; fi # skip if
already mounted
27         cand="$name"; break
28     done < <(lsblk -dno NAME,TYPE)
29
30     if [ -n "$cand" ]; then
31         dev="/dev/$cand"
32         echo "[scratch] spare disk: $dev"
33         if ! blkid "$dev" >/dev/null 2>&1; then
34             echo "[scratch] no filesystem -> mkfs.ext4 $dev"
35             mkfs.ext4 -F -q "$dev"
36         fi
37         mountpoint -q "$MNT" || mount "$dev" "$MNT"
38         chmod 1777 "$MNT"
39         echo "[scratch] mounted $dev at $MNT ? $(df -h "$MNT" | awk 'NR==2{print $2}' total,
"$4" free}')"
40     else
41         echo "[scratch] no spare NVMe found ? using $MNT on the boot disk (fine for a CPU
box)"
42         chmod 1777 "$MNT"
43     fi
44
45     echo
46     echo ">> Point the pipeline at scratch: export TMPDIR=$MNT (or pass --scratch $MNT to
backend.worker)"
47

```

14. assistant (2026-06-20T08:22:45.864Z)

Perfect. Now I have all the documentation. Let me synthesize the key findings into sections for your GCP pivot.

Synthesis Report: M3.5 Linux-native WASB DetectorRunner & GCP Pivot

1. **M3.5 Specification & Acceptance Criteria** (04-BUILD-AND-TEST-PLAN.md)

```

What M3.5 is:
- Stand up a `LinuxNativeRunner` behind the existing `DetectorRunner` ABC
([`backend/pipeline/detectors/base.py`](backend/pipeline/detectors/base.py)) ? the main de-risk
goal
- Skips all `wsl`/`conda activate`/staging; loads WASB/TrackNet weights natively (torch/TF) on
Linux
- Writes the identical `Frame,Visibility,X,Y` CSV so downstream (`parse_trajectory_csv`,
`trajectory_to_action_windows`) runs byte-for-byte identical
- WSL adapters stay as one backend of the ABC; Windows dev behavior is 100% preserved

```

```

**Acceptance test (the parity guarantee):**
- Same clip through **WSL backend (Windows)** and **LinuxNativeRunner` (D0/Linux)** ?
`Frame,Visibility,X,Y` CSVs match within tolerance AND identical candidate windows
- WASB/TrackNet hybrid segmenter produces same rallies on D0 as on WSL box
- **M3.5 doubles as `DESKTOP_APP_PLAN` P0 feasibility spike**

**Status:** PROPOSAL · M3.5 resolved 2026-06-20: **$200 credit covers Paperspace GPUs + free unbilled M4000** (verify M4000's `sm_52`/Maxwell arch is in pinned torch build)

```

2. ****Risks (E1?E6) & Weight Provenance** (06-RISKS-AND-OPEN-QUESTIONS.md)**

****Ranked technical risks:****

#	Risk	Mitigation
E1	WSL2 bridge is load-bearing; Linux-native path **never run** Prove via **M3.5 parity test on D0** (Ubuntu+CUDA = native conda env, minus WSL); WASB's shipped badminton weights make real trajectory testable today	
E2	TF + torch in one image segfault Two single-framework images sharing `/scratch`; **never co-install TF + torch**	
E3	\$200 budget only holds if Gen-2 VLM excluded ? Resolved: Gen-0 only (numpy, ~4.5 GB VRAM, minutes)	
E4	Free-credit myths **GCP \$300/90-day is the only clean win** ; AWS/Azure splits; D0 has real \$200 but lacks serverless-GPU metering	
E5	Data-contract break via re-encode MD5 keys the join; collector's proxy transcode changes it; **add `md5` to collector export + decide original-vs-proxy keying** (M2 prerequisite)	
E6	Spot preemption loses local disk Checkpoint train state to bucket every ?30 min; inference jobs are retry-safe	

****WASB-C3 (weight provenance):****

```

- `wasb_badminton_best.pth.tar` pretrained weights' license/source-dataset provenance is **NOT verified** in setup scripts
- **Clear before sharing any derived trajectories or model**; fine for internal extraction + Gen-0 bootstrapping

```

3. ****Compute Backend Abstraction & GPU Runner** (07-COMPUTE-ABSTRACTION.md)**

****Key architectural axes (keep separate):****

...

WHERE job runs (ComputeBackend)	?	HOW GPU stage executes (DeviceContext)
local / D0 droplet / serverless	?	probe actual GPU ? resolve batch/precision/chunk

****ComputeBackend registry**** ? not yet a single interface, but:

```

- `ComputeBackend` answers which machine (local | hosted | byo_worker | serverless)
- **DeviceContext** answers how to run GPU work on that machine (spec-aware, per-stage)

```

****GPU/CPU segmentation rule:****

```

- **GPU-bound:** Person detection (torchvision), **Shuttle trajectory (WASB torch / TrackNet TF)** ? produce `Frame,Visibility,X,Y` CSV
- **CPU:** Validators, ffmpeg chunking, ByteTrack, windowing, AI handoff (Gemini), DB writes
- **Device-agnostic downstream:** Everything after artifact boundary

```

****Proposed device layer (implement at M3.5, not M0):****

```

```python
@dataclass(frozen=True)
class DeviceSpec:
 kind: str # "cuda" | "mps" | "cpu"
 name: str | None # "NVIDIA L4" from run_telemetry.gpu_name()

```

```

 vram_gb: float | None
 compute_capability: tuple[int,int] | None
 supports_fp16, supports_bf16: bool
 num_devices: int

class DeviceContext:
 def to(self, model_or_tensor): ... # placement (cuda/mps/cpu)
 def autocast(self): ... # precision ctx
 def run_with_oom_retry(self, fn, halve="batch_size"): ...
 @classmethod
 def detect(cls, config) -> "DeviceContext": # probe spec ? resolve profile
 ...

Provider-grounded `DeviceProfile` registry (M3.5 scope):
- **DO/Paperspace (now):** M4000 8GB (free, Maxwell `sm_52`), P4000/RTX4000, A4000 16GB,
A5000/A6000, L40S, A100
- **GCP (later):** T4 16GB, L4 24GB (Cloud Run GPU), A100
- **Fallback:** `HeuristicProfile` derives from VRAM + compute capability

Resolution order (most specific wins):
1. Explicit override (`config.indexing.device.{batch_size,precision,?}`)
2. Named profile (e.g., `L4_24GB`)
3. Heuristic from `DeviceSpec`
4. Conservative CPU defaults

```

#### 4. **Detector/Runner Contract** (02-COMPUTE-MAP-AND-CONTRACT.md & README.md)

```

The one data contract everything keys on:
...

rally-annotator ? sports-data-collector ? badminton-highlight-indexer
<video>.rallies.csv SQLite ? manifest.json keyed by file MD5
...

- **Identity:** Indexer keys videos by whole-file MD5 (`compute_video_id`, 64 KB-chunked);
collector keys by human `video_id`
- **Intervals:** decimal seconds `[start, end)`; ending-reason taxonomy
(winner/forced_error/unforced_error/service_fault/let/other)
- **Train/serve wall (ADR-008):** artifact bundle (weights + manifest), never a code import;
content/semver-keyed

M3.5 DetectorRunner ABC signature:
- Base: `backend/pipeline/detectors/base.py` (run_predict/healthcheck abstract tasks)
- WSL plumbing: separate `WslCommandMixin` (native runner must NOT inherit)
- **Parity contract:** any runner emitting `Frame,Visibility,X,Y` CSV reuses downstream verbatim
- Windows dev untouched; env/OS selects the runner at instantiation

WASB/TrackNet on DO (today's path):
- Runs natively via conda env (no WSL needed on Linux)
- WASB: PyTorch 1.11 + CUDA 11.3 (torch 1.11.0+cu113)
- TrackNet: TensorFlow 2.17 (bundled CUDA 12.3)
- Output: identical `Frame,Visibility,X,Y` CSV ? existing `parse_trajectory_csv` +
`trajectory_to_action_windows` downstream

```

#### 5. **GPU Provisioning & Path Pinning ? DigitalOcean & GCP** (SETUP\_NEW\_MACHINE.md §2, §7b, §9)

```

DigitalOcean GPU Droplet provisioning (current, to be migrated):
``bash
doctl compute droplet create rally-gpu --image gpu-h100x1-base --size gpu-4000adax1-20gb \
 --region tor1 --ssh-keys <your-key-fp> --wait
...

- ? "GPU regions" on DO: `nyc2`/`tor1`/`atl1` ? DO has NO Singapore GPU droplet (sgp1 is
```

CPU-only)

- Dual local NVMe; RTX 4000 Ada (20 GB) specified

**\*\*Critical path contradiction you must resolve:\*\***

- **\*\*Doc says:\*\*** "M3.5 parity test on DigitalOcean" (04-BUILD-AND-TEST-PLAN.md §§3.5, 06 §1 mitigation, README §59)

- **\*\*Reality:\*\*** DO has NO GPU region in sgp1/Singapore; GPU regions are `nyc2`/`tor1`/`atl1` (North America/Toronto)

- **\*\*GCP alternative:\*\*** `asia-southeast1` (Singapore) and `asia-south1` (Mumbai) have A100, L4 GPUs

**\*\*GCP Compute setup (your pivot target):\*\***

```
```bash
```

From SETUP_NEW_MACHINE.md §7b (GCS backend config):

```
export GOOGLE_APPLICATION_CREDENTIALS=/path/to/sa.json
```

config.json:

```
"storage": { "backend": "local", "gcs": { "project": "<gcp-project-id>" } }
```

Worker command:

```
env -u GEMINI_API_KEY RALLY_DETECTOR_IMPL=stub python -m backend.worker infer \
  --video-uri gcs://<bucket>/videos/clip.mp4 --out-uri gcs://<bucket> \
  --segmenter wasb_hybrid --set indexing.skip_ai_handoff=true
...
```

****GCP-specific config keys (live in code as verified 2026-06-20):****

- `storage.gcs.project` ? GCP project ID (no endpoint, uses googleapis.com)
- `GOOGLE_APPLICATION_CREDENTIALS` ? path to SA JSON (Storage Object Admin role)
- No `endpoint_url` needed (defaults to GCP)

6. ****WASB Conda Environment & Native Setup**** (TRACKNET_WSL_SETUP.md & gpu_setup.sh)

****Exact conda env name (Linux-native):**** `wasb` (Python 3.8)

****WASB-SBDT repo path:****

```
```bash
```

```
MODELS_DIR="${RALLY_MODELS_DIR:-$HOME/models}"
```

```
WASB_REPO="$MODELS_DIR/WASB-SBDT" # ? ~/models/WASB-SBDT
```

```
```
```

- Clone: `git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO"`

****Weights landing path:****

```
```bash
```

```
WEIGHTS="$WASB_REPO/pretrained_weights/wasb_badminton_best.pth.tar"
```

## Expected: ~/models/WASB-SBDT/pretrained\_weights/wasb\_badminton\_best.pth.tar

```
```
```

- ? If missing, script warns: ``"!! MISSING $WEIGHTS ? fetch wasb_badminton_best.pth.tar (Drive/bucket) before extraction"`

- ****This is NOT fetched by setup; you must pre-stage it**** (Drive/bucket, M3.5 prerequisite)

****Env activation (Linux-native, M3.5):****

```
```bash
```

```
source $CONDA_DIR/etc/profile.d/conda.sh && conda activate wasb
```

```
```
```

****Device selection (WASB native):****

```
```bash
```

## In gpu\_setup.sh §5:

```
python - <<'PY'
```

```
import torch
```

```
ok = torch.cuda.is_available()
```

```

assert ok, "CUDA not available in the wasb env ? check GPU driver / base image"
PY
...
- Assertion fails if GPU not visible; script sets up the env to have
`torch.cuda.is_available()==True`
- **No hardcoded device selection** in setup; WASB's `device='cuda'` is the old behavior (M3.5
replaces with `DeviceContext.to()`)

WASB stack (hardcoded in gpu_setup.sh):
```bash
pip install torch==1.11.0+cu113 torchvision==0.12.0+cu113 \
  --extra-index-url https://download.pytorch.org/whl/cu113
[ -f "$WASB_REPO/requirements.txt" ] && pip install -q -r "$WASB_REPO/requirements.txt" || true
```

- PyTorch 1.11.0 + CUDA 11.3 (old stack; ? main indexer's 2.12+ cu124)
- torchvision 0.12.0+cu113

Detector inference flow (WasbRunner in indexer):
- Backend module: `backend/pipeline/detectors/wasb_infer.py` (staged into conda env, runs
detector+tracker, emits CSV)
- Segmenter: `backend/pipeline/segmenters/trajectory_hybrid.py` (consumes CSV via `wasb_hybrid`)
- Config keys: `indexing.wasb.*` (see CODE_MAP §2.1/§2.2; no hardcoded weights path in indexer ?
config-driven)

```

## 7. \*\*Bootstrap, Mount, & Env Setup Scripts\*\* (deploy/digitalocean/\*.sh)

```

Canonical spin-up sequence:

$7a) mount_scratch.sh ? NVMe scratch disk
```bash
sudo bash deploy/digitalocean/mount_scratch.sh && export TMPDIR=/scratch
```

- Detects spare NVMe (skip root disk + tiny drives <10 GB)
- Formats `mkfs.ext4` if needed, mounts at `/scratch`
- Chmod 1777 (world-writable, ephemeral)
- Falls back to `/scratch` on boot disk if no spare NVMe found
- **NOT added to `/etc/fstab`** (deliberately ephemeral; device rename could block boot)

$7b) bootstrap.sh ? Python venv & torch
```bash
./deploy/digitalocean/bootstrap.sh          # CPU box
./deploy/digitalocean/bootstrap.sh --gpu    # GPU box
```

- Installs: `python3 python3-venv python3-pip git ffmpeg libgl1 libglib2.0-0t64`
- Creates `.venv`, upgrades pip
- **CPU path:** `pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu`
- **GPU path:** `pip install torch torchvision --index-url
https://download.pytorch.org/whl/cu124`
 - ? **Paperspace M4000 (Maxwell `sm_52`) warning:** "old archs sometimes dropped; if
`torch.cuda.is_available()` is False, use P4000/RTX4000 or pin older torch"
- Editable install: `pip install -e .[dev]`
- Prints device check: `torch.cuda.is_available()`

$7c) gpu_setup.sh ? Native WASB env (M3.5)
```bash
bash deploy/digitalocean/gpu_setup.sh
```

- Creates separate conda env `wasb` (Python 3.8)
- Clones WASB-SBDT repo to `$MODELS_DIR/WASB-SBDT` (~$HOME/models/WASB-SBDT)
- Installs: `torch==1.11.0+cu113 torchvision==0.12.0+cu113`
- Checks for weights at `$WASB_REPO/pretrained_weights/wasb_badminton_best.pth.tar`
- Verifies `torch.cuda.is_available()==True`
- **Does NOT fetch weights** ? you must pre-stage

```

- **\*\*Does NOT set any env vars\*\*** (conda env activation is manual)

**\*\*Env vars NOT set by scripts (you must set manually):\*\***

- `TMPDIR=/scratch` ? pipeline reads this for fast decode
- `WASB\_REPO` or `RALLY\_MODELS\_DIR` ? only used in the setup script, not runtime
- `GOOGLE\_APPLICATION\_CREDENTIALS=/path/to/sa.json` ? GCS backend (not DigitalOcean)
- `GEMINI\_API\_KEY` ? injected as secret, not baked (see SETUP\_NEW\_MACHINE.md §6b)
- `AWS\_ACCESS\_KEY\_ID` / `AWS\_SECRET\_ACCESS\_KEY` ? S3/DO Spaces backend (stored in `~/.aws/credentials`)

**\*\*Full GPU box spin-up (copy-paste from §9):\*\***

```
```bash
cd ~/rally
sudo bash deploy/digitalocean/mount_scratch.sh && export TMPDIR=/scratch
./deploy/digitalocean/bootstrap.sh --gpu
. .venv/bin/activate && pip install -e .[storage-s3]
bash deploy/digitalocean/gpu_setup.sh
```

Secrets (non-reproducible):

```
mkdir -p /root/.keys && printf '%s' '<rotated-key>' > /root/.keys/GEMINI_API_KEY && chmod 600
/root/.keys/GEMINI_API_KEY
install -m600 /dev/stdin ~/.aws/credentials <<'INI'
[default]
aws_access_key_id = <key>
aws_secret_access_key = <secret>
INI
---
```

8. ****Critical GCP Pivot Contradictions & Resolutions****

```
| Contradiction | DO Status | GCP Status | Your Action |
|---|---|---|---|
| **M3.5 parity test on DO** (04 §3.5, 06 mitigation, README line 98) | DO has NO GPU in sgp1 (Singapore); GPU only in `nyc2`/`tor1`/`atl1` | GCP asia-southeast1 (SG) & asia-south1 (Mumbai) have A100, L4 GPUs native | **Pivot M3.5 parity test to GCP asia-southeast1 or asia-south1** |
| **DO $200 credit** (04, 06 decision #2) | DO credit covers CPU Droplets + RTX 4000 GPU in NYC/Toronto, **not** GCP | GCP $300/90-day trial (clean, non-split) for asia-southeast1 test | **Use GCP trial for M3.5 parity proof; can revisit DO for prod** |
| **Storage egress** (06 decision #3 recommends DO Spaces) | DO Spaces: 1 TiB egress included, **sgp1 region ties to CPU-only Droplets** | GCS: zero cross-region egress if worker in asia-southeast1; native to GCP | **Use GCS for GCP; DO Spaces only if you stay on DO droplets** |
| **Serverless metering** (06 decision #2, 03 §4) | DO lacks true serverless GPU ? coarse per-droplet wall-clock billing | GCP Cloud Run GPU + GCP Compute Engine: native per-invocation GPU-seconds | **GCP's Cloud Run GPU is the M3 endpoint** (exact per-video bill) |
```

9. ****Exact CLI Commands (Copy-Paste for GCP Pivot)****

****GCP GPU VM provisioning (M3.5 parity test, asia-southeast1):****

```
```bash
```

## Create GPU VM (A100 or L4 ? check region availability):

```
gcloud compute instances create rally-gpu-sg \
 --zone asia-southeast1-a \
 --machine-type a2-highgpu-1g \
 --image-family=ubuntu-2404-lts \
 --image-project=ubuntu-os-cloud \
 --accelerator type=nvidia-a100,count=1 \
 --maintenance-policy=TERMINATE \
 --scopes default,compute-rw,storage-rw \
 --metadata startup-script='#!/bin/bash\ncurl
https://raw.githubusercontent.com/GoogleCloudPlatform/compute-gpu-
```

```
installation/main/linux/setup_gpu.sh | bash'
...

```

```
Get code onto GCP box (same as D0):
```bash

```

Option A ? git clone (with PAT):

```
gcloud compute ssh rally-gpu-sg --zone asia-southeast1-a -- \
  git clone https://<PAT>@github.com/avidullu/badminton-highlight-indexer.git ~/rally

```

Option B ? tar+scp:

```
git archive --format=tar.gz -o /tmp/rally.tgz HEAD
gcloud compute scp /tmp/rally.tgz rally-gpu-sg:/root/ --zone asia-southeast1-a
gcloud compute ssh rally-gpu-sg --zone asia-southeast1-a -- \
  'mkdir -p ~/rally && tar -xzf ~/rally.tgz -C ~/rally'
...

```

```
**Bootstrap on GCP (same commands as D0):**
```bash
gcloud compute ssh rally-gpu-sg --zone asia-southeast1-a -- \
 'cd ~/rally && sudo bash deploy/digitalocean/mount_scratch.sh && export TMPDIR=/scratch && \
 ./deploy/digitalocean/bootstrap.sh --gpu && \
 bash deploy/digitalocean/gpu_setup.sh'
...

```

```
GCS backend setup (M3.5 inference test):
```bash

```

On GCP box, create/download SA JSON:

```
gcloud iam service-accounts create rally-worker --project=$PROJECT_ID
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member=serviceAccount:rally-worker@$PROJECT_ID.iam.gserviceaccount.com \
  --role=roles/storage.objectAdmin
gcloud iam service-accounts keys create /root/sa.json \
  --iam-account=rally-worker@$PROJECT_ID.iam.gserviceaccount.com

```

Set env:

```
export GOOGLE_APPLICATION_CREDENTIALS=/root/sa.json

```

config.json (GCS backend):

```
cat > ~/rally/config.json <<'JSON'
{
  "sport": "badminton",
  "indexing": { "default_segmenter": "wasb_hybrid", "detector_impl": "stub", "skip_ai_handoff":
true },
  "storage": { "backend": "local", "gcs": { "project": "your-project-id" } }
}
JSON

```

Run worker infer:

```
. .venv/bin/activate && conda activate wasb # both envs
python -m backend.worker infer \
  --video-uri gcs://your-bucket/videos/test.mp4 \
  --out-uri gcs://your-bucket \
  --segmenter wasb_hybrid --set indexing.skip_ai_handoff=true
...

```

```
**Teardown (GCP):**
```bash
gcloud compute instances delete rally-gpu-sg --zone asia-southeast1-a --quiet
gcloud iam service-accounts delete rally-worker@$PROJECT_ID.iam.gserviceaccount.com --quiet
...

```



---

## 10. **\*\*Summary: M3.5 & GCP Readiness Checklist\*\***

**\*\*Before you build:\*\***

- [ ] **\*\*Resolve M3.5 location:\*\*** M3.5 parity test must run on **\*\*GCP asia-southeast1-a or asia-south1-a\*\*** (not DO sgp1, which lacks GPU). Use GCP \$300 trial.
- [ ] **\*\*WASB weights:\*\*** Pre-stage ``wasb_badminton_best.pth.tar`` in bucket or ``/root/models/WASB-SBDT/pretrained_weights/`` on the GCP VM (`gpu_setup.sh` doesn't fetch it).
- [ ] **\*\*Conda env isolation:\*\*** ``wasb`` env (py3.8, torch 1.11+cu113) is SEPARATE from main indexer venv (py3.10+, torch 2.12+cu124); never co-install.
- [ ] **\*\*DeviceContext (M3.5 code):\*\*** Implement ``DeviceSpec`` + ``DeviceContext.to()`` to replace WASB's hardcoded ``device='cuda'``; inject into ``WasbRunner`` at instantiation.
- [ ] **\*\*GCS config keys:\*\*** ``storage.gcs.project`` only (no endpoint); ``GOOGLE_APPLICATION_CREDENTIALS`` env var for auth.
- [ ] **\*\*Scratch mount:\*\*** GCP VMs will need ``mount_scratch.sh`` adapted (GCP Persistent Disks vs DO NVMe detection differs).
- [ ] **\*\*M3.5 parity test:\*\*** Same clip on Windows WSL backend & GCP native ``LinuxNativeRunner`` ? byte-identical ``Frame,Visibility,X,Y`` CSVs.

**\*\*Exact build order (from docs):\*\***

1. **\*\*M0 (CPU, \$0):\*\*** ``pytest -q`` on the bootstrap venv (no GPU, no WSL)
2. **\*\*M1 (GPU smoke test):\*\*** ``nvidia-smi`` + single ``infer`` on a real clip, faster than M0
3. **\*\*M2 (bucket train):\*\*** Pull corpus via ``rclone``, run Gen-0, push versioned weights
4. **\*\*M3.5 (native detector parity ? YOUR PIVOT):\*\*** GCP GPU ? native conda ``wasb`` ? identical trajectories vs WSL
5. **\*\*M3 (inference endpoint):\*\*** Cloud Run GPU or RunPod Serverless for per-video metering
6. **\*\*M4 (scale-to-zero + spend cap):\*\*** Idle billing \$0, rate limits, spend alerts

---

**\*\*Final caveat:\*\*** The docs consistently say "M3.5 on DigitalOcean" (04 line 98, 06 title/mitigation, README line 59). **\*\*You are resolving this contradiction by pivoting to GCP.\*\*** Document this decision in ``docs/archives/decisions/DECISIONS.md`` so future maintainers know why M3.5 runs on GCP asia-southeast1, not DO sgp1.